

Table of Contents

Page Numbers

1. Introduction	---	3
1.1 Purpose		
1.2 Intended Customers/Users		
1.3 Product Scope		
1.4 Outline of the document		
2. Overall Description	---	4
2.1 Product Perspective		
2.2 Product Functions		
2.3 User Classes		
2.4 Design and Implementation Constraints		
3. Diagrams		
3.1 UML Diagrams		5 – 25
3.2 ER Diagrams		
3.3 DFD Diagrams		
3.4 Assumptions and Dependencies		
4. External Interface Requirements/Screenshots		
4.1 User Interfaces		
4.2 Hardware Interfaces		
4.3 Software Interfaces		
4.4 Communication Interfaces		
5. Other Non-Functional Requirements		
5.1 Performance Requirements		
5.2 Safety Requirements		
5.3 Security Requirements		
5.4 Software Quality Attributes		
5.5 Business Policies/Rules (if any)		
6. Testing		
6.1 Block-Box Testing		
6.2 White-Box Testing		
7. Conclusion		
8. References		
9. Acknowledgements		

Chapter 1 – Introduction

1.1 Purpose

This system helps both gym owners and customers to keep record of everything, the main goal is to replace manual record-keeping with a digital solution that efficiently manages gym users data, attendance, and fee collection(will send notifications if the membership is expiring). Uniquely, this system also serves a critical maintenance function by tracking the health status of gym equipment(by checking the warranty given for the equipment's), scheduling repairs, and minimizing danger to ensure member safety , this will help to get reduce human hours and maintain records properly.

The purpose of this document is to show the requirements for this system and also specify how it is going to workl.

1.2 Intended Customers /Users

Gym Owner: to see total revenue and overall business status.

Receptionist: to registers new members, collects fees, and marks attendance.

Maintenance staff: Staff responsible for repairing equipment (they will get an alarm if some gym equipment gets damaged like treadmill or bench-press).

1.3 Product Scope

The Gym Maintenance System is a desktop application. Currently, many gyms use registers or Excel sheets to track members (Even the Gym in SAC building), which leads to errors and lost data. This system will digitize those records. Our system helps us in :

- **Member Management:** Adding new members, updating their weight and height, and tracking membership expiry dates.
- **Equipment Maintenance:** A specific module to log details of machines (e.g., Treadmill, Bench Press), track their service dates, and flag them as "Under Maintenance" if broken and send a notification to maintenance staff.
- **Fee Tracking:** Recording who has paid and who has pending dues.
- **Reports:** Generating simple reports for monthly income and equipment health status.
- **Staff attendance:** to check whether the cleaners and security attended or not , if not make a final report for month end .

1.4 Outline of the Document

This helps us to keep track of overall project , the primary purpose of this document is to tell how the project works and what all we have used and what have we done for it ,it will keep the record of what designs we have used ,what references we have used and specifies the tests we have done .

Chapter 2 – Overall Description

2.1 Product Perspective

This project is a desktop based software application developed to help both gym owners and members . It is designed to replace the manual "logbook" system currently used in all the gyms . It stores data in a local or server-based database (or can maintain in a local hard disk) . It does not currently integrate with external payment gateways or biometric devices in this version here data entry is manual via keyboard . In future scope will try to integrate them.

2.2 Product Functions

The major functions of the system are:

- **authentication:** Secure login for Admin and Staff.
- **Registration:** Form interface to enter Member Name, Contact, Address, and Plan selection.
- **Maintenance Log:** A feature to select a machine and update its status (Working / Needs Repair / Repaired).
- **Search:** Ability to find a member by their Name or Phone Number.
- **Dues Alert:** Visual highlight for members whose fees are overdue.

2.3 User Classes and Characteristics

Administrator: Has full permissions. Can delete member records, view financial data, and add staff accounts. Expected to have basic computer knowledge.

Staff: Restricted permissions. Can add members and mark attendance but cannot delete data or see total profit. They may have limited technical skills, so the UI must be simple.

Gym Members: They do not use the system directly; they interact with the Front Desk Staff who input their data.

2.4 Design and Implementation Constraints

Hardware: The system must be lightweight enough to run on a standard Windows PC (i3 processor, 4GB RAM) which is common in gym reception areas.

Input: for now we didn't implement scanners or biometric , the input should be given by the keyboard that is a major constraint .

Design: it should be interactive and easy to handle front page with images and good animations so the user can handle easily , the blocks and grid placement of the front page must be managed properly so even a person with basic knowledge can access .

Chapter 3 – Diagrams

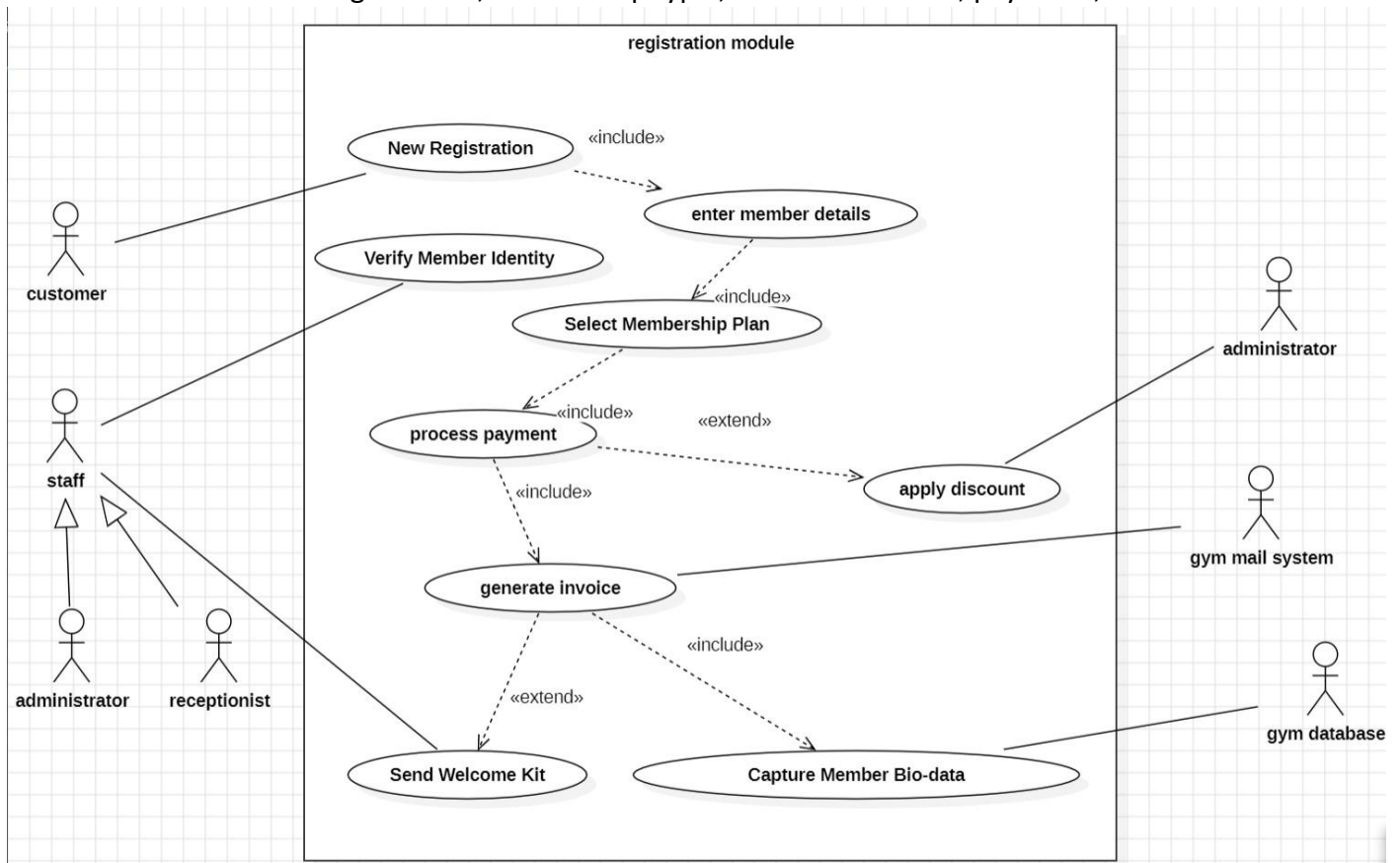
3.1 UML Diagrams

Use Case Diagrams

a) User and membership management

Actors : customer , staff , gym database , gym mail system , administrator .

Use cases : registration , membership type , validate member , payment , invoice .

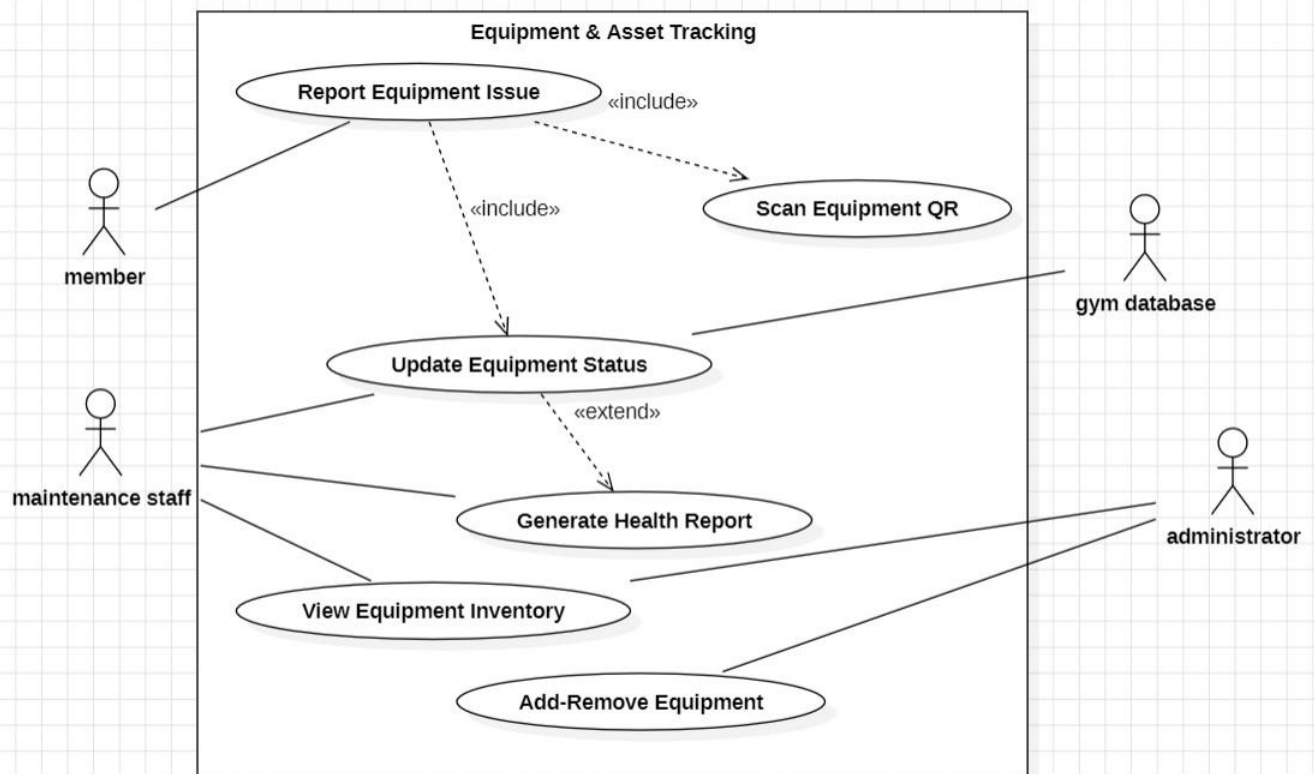


Relations : Register Member-> Enter Personal Details-> Verify Identity-> Select Membership Type-> Apply Discount Code-> Generate Invoice-> Send Enrollment Confirmation

b) Equipment & Asset Tracking

Actors : customer , staff , gym database , administrator.

User cases : View Equipment Inventory ,Add/Remove Equipment, Update Equipment Status, Report Equipment Issue, Generate Health Report, Scan Equipment QR

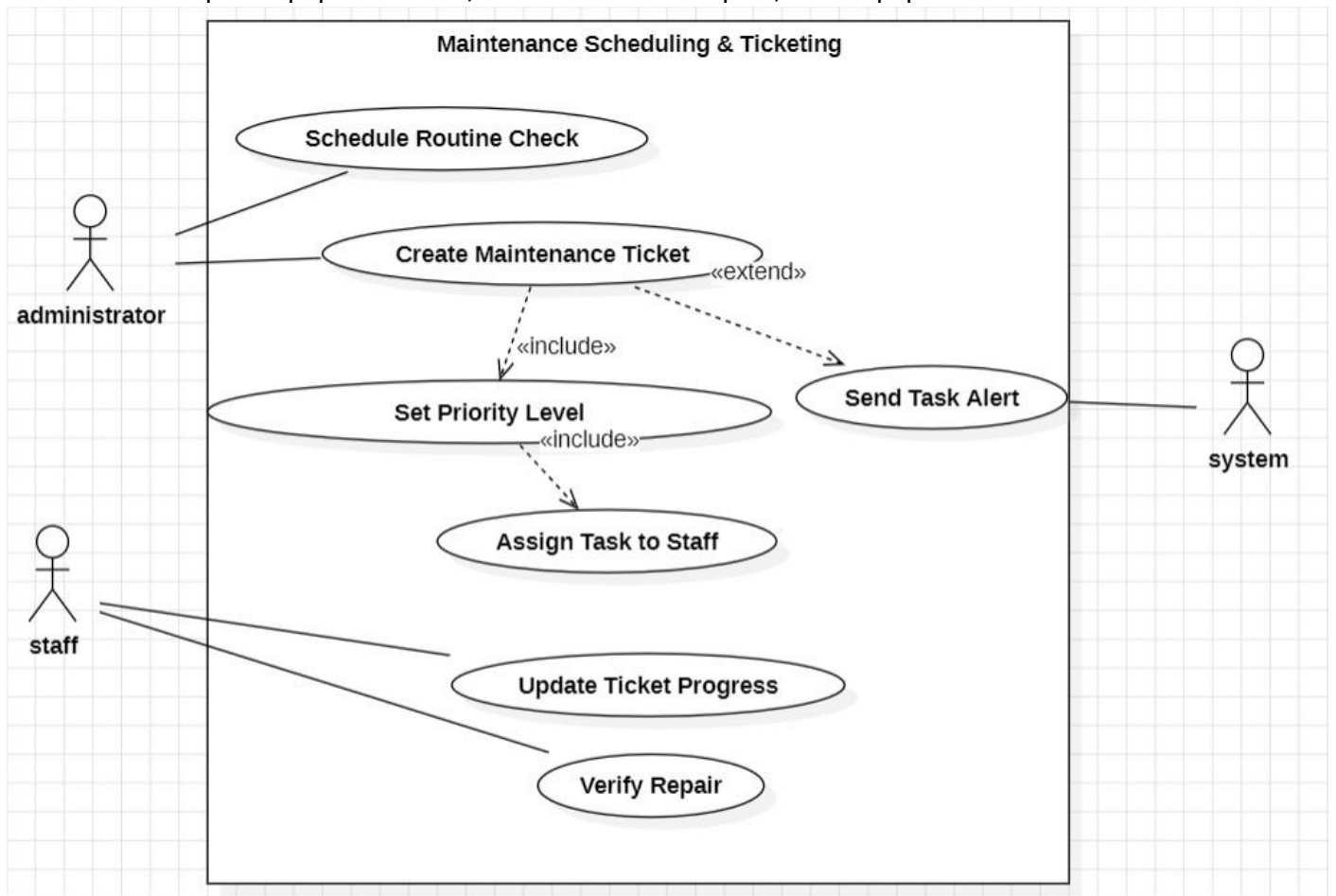


Relations : Report Equipment Issue -> Scan Equipment QR -> Update Equipment Status-> Generate Health Report-> View Equipment Inventory-> Add/Remove Equipment

c) Maintenance Scheduling & Ticketing

Actors : system , staff , administrator.

User cases : View Equipment Inventory ,Add/Remove Equipment, Update Equipment Status, Report Equipment Issue, Generate Health Report, Scan Equipment QR

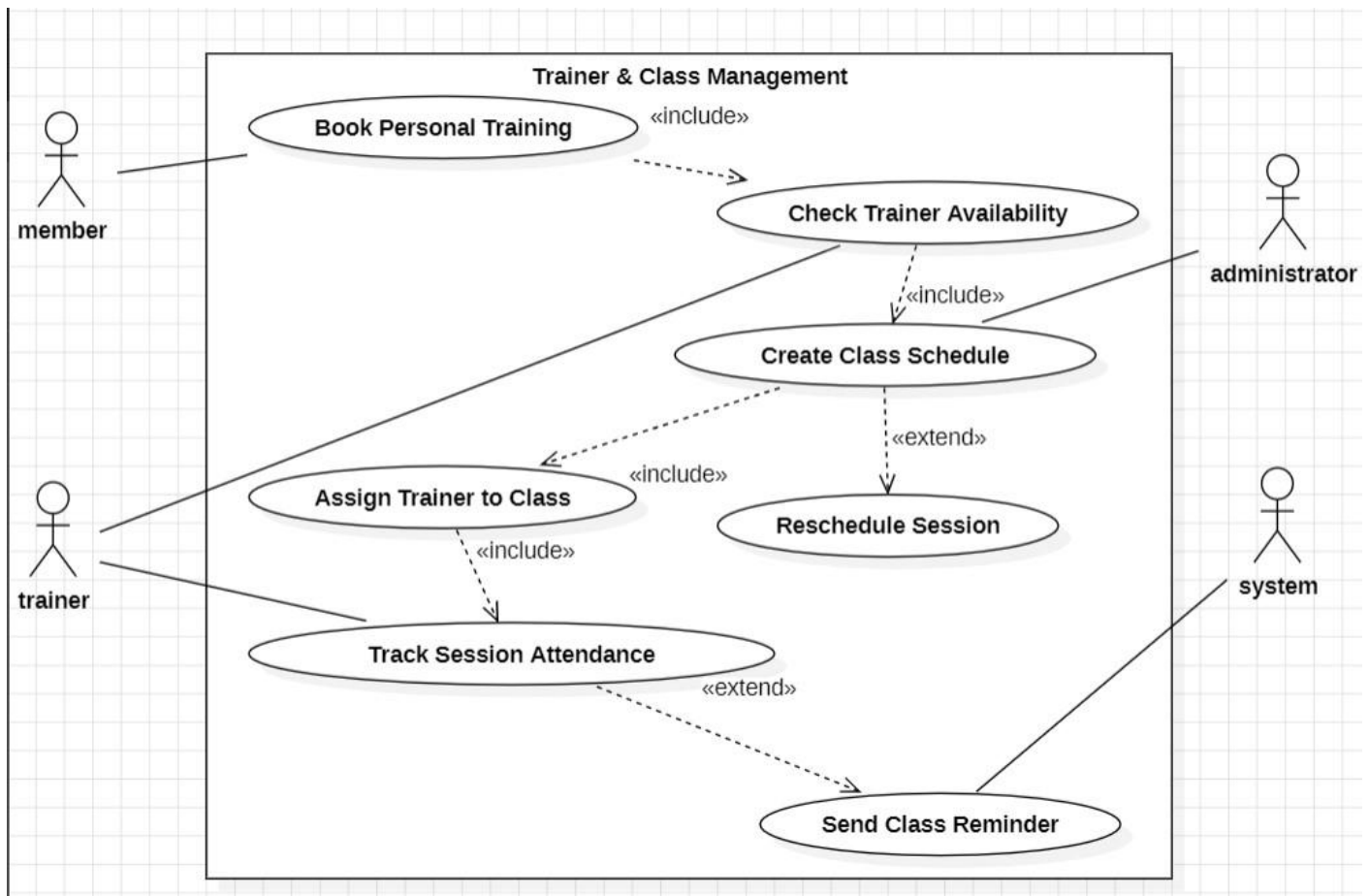


Relations : Schedule Routine Check -> Create Maintenance Ticket -> Set Priority Level -> Assign Task to Staff -> Send Task Alert -> Update Ticket Progress -> Verify Repair

d) Trainer & Class Management

Actors : system , trainer , member , administrator.

User cases : Book Personal Training, Book Personal Training, Assign Trainer to Class, Check Trainer Availability, Track Session Attendance, Cancel/Reschedule Session, Send Class Reminder.

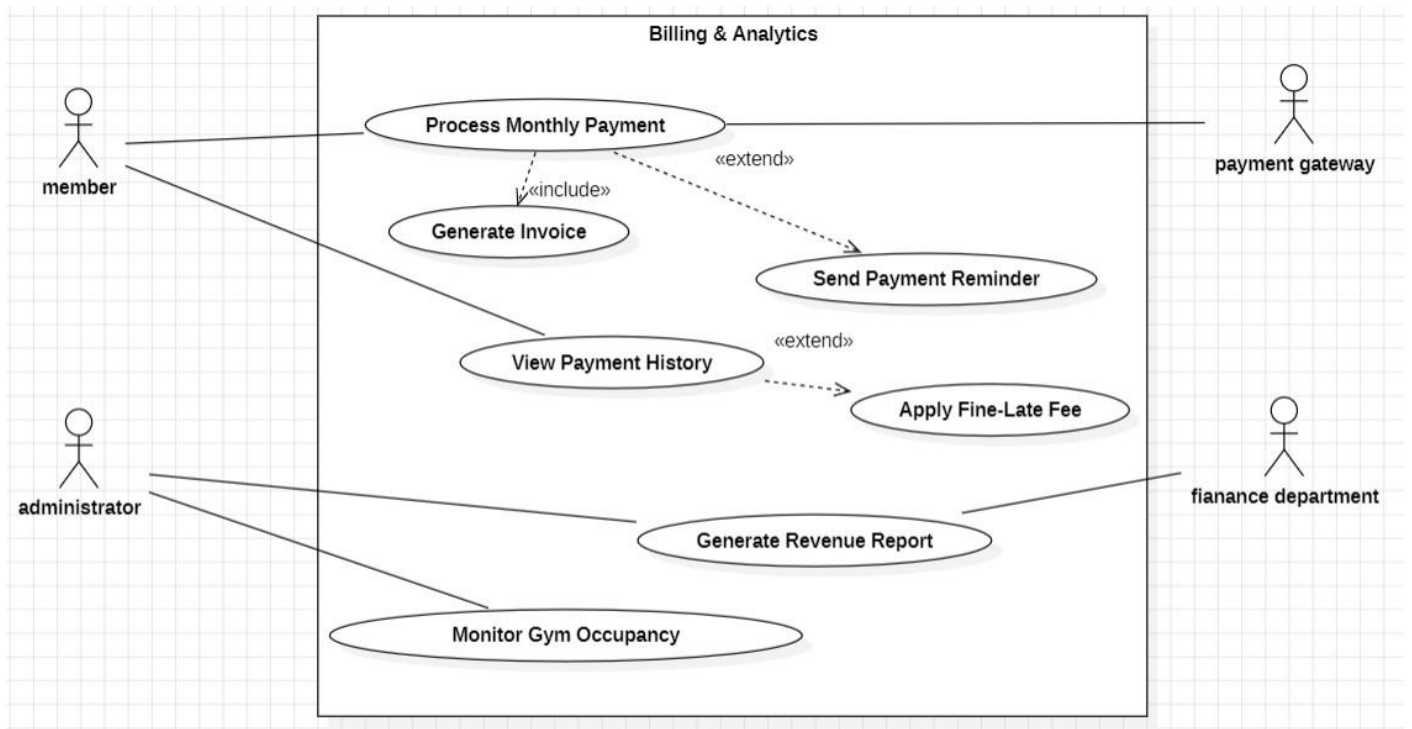


Relations : Create Class Schedule -> Assign Trainer to Class -> Check Trainer Availability -> Book Personal Training -> Send Class Reminder -> Track Session Attendance

e) Billing & Analytics

Actors : member , administrator , payment gateway , finance department.

User cases : Process Monthly Payment, Generate Invoice, Apply Fine/Late Fee, View Payment History, Generate Revenue Report, Monitor Gym Occupancy, Send Payment Reminder

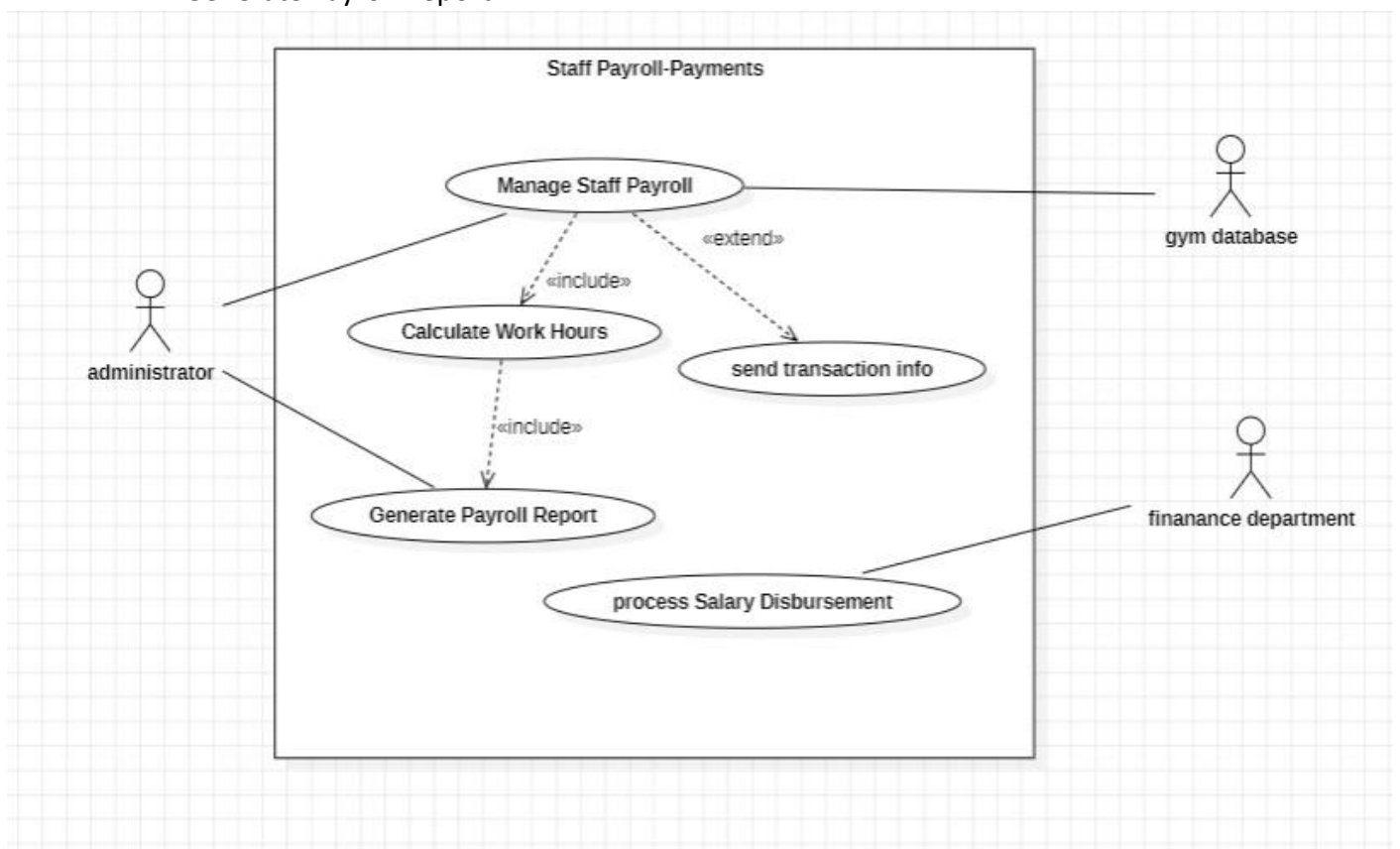


Relations : Send Payment Reminder -> Process Monthly Payment -> Generate Invoice -> View Payment History -> Apply Fine/Late Fee -> Generate Revenue Report -> Monitor Gym Occupancy

f) Staff Payroll/Payments

Actors : database, administrator , finance department.

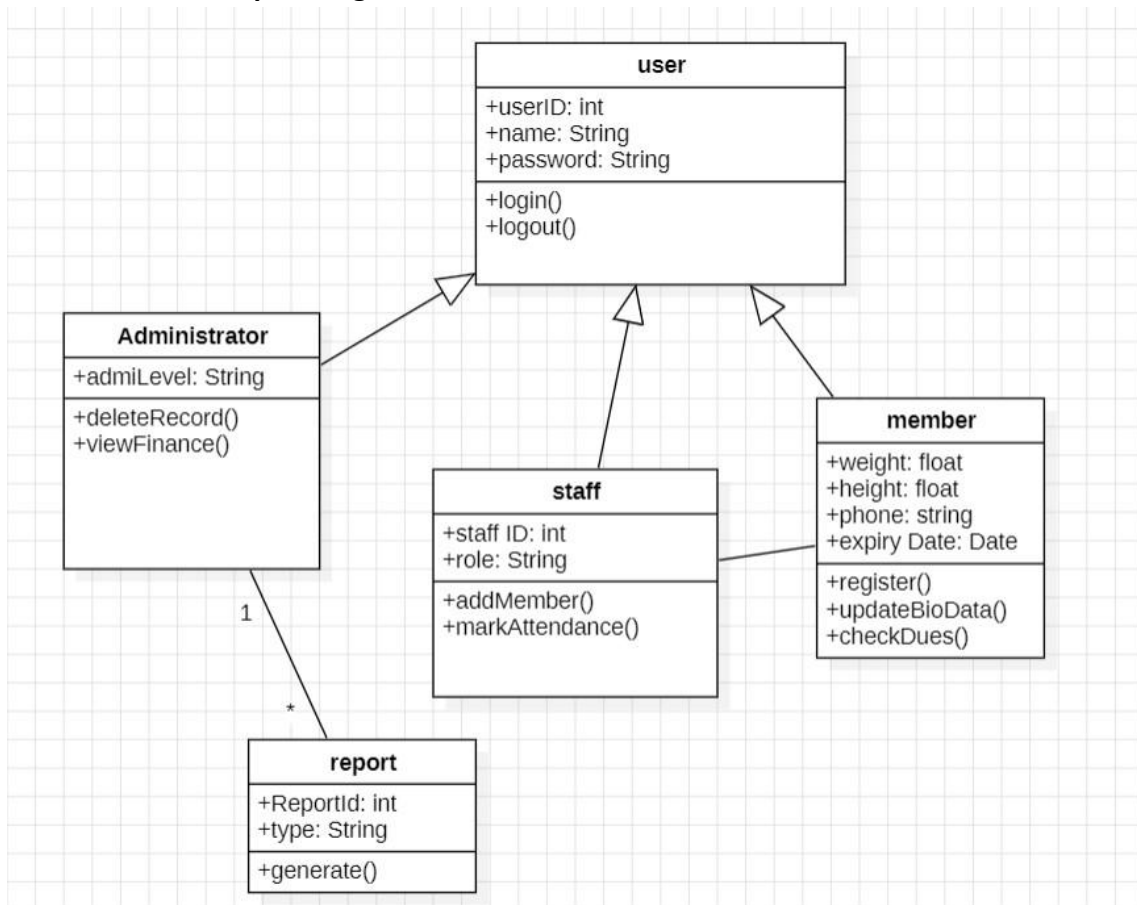
User cases : Manage Staff Payroll, Calculate Work Hours, Process Salary Disbursement, Generate Payroll Report



Relations : Calculate Work Hours -> Manage Staff Payroll -> Process Salary Disbursement
- > Generate Invoice -> Generate Payroll Report

- Class diagrams

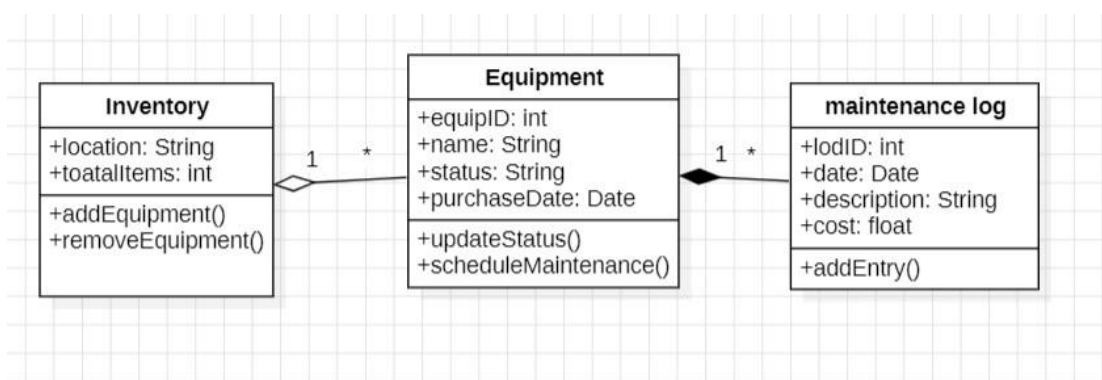
a) User and membership management



Logic: It uses a central User class to handle common login and logout functions for all system actors. The Member class is specialized to track fitness metrics like weight, height, and membership expiry. **Relationships:**

- Inheritance: Administrator , Staff, and Member classes inherit from the base User class using a generalization relationship .they all are users of the system.
- Multiplicity: One Administrator can manage or view many Reports

b) Equipment & Asset Tracking

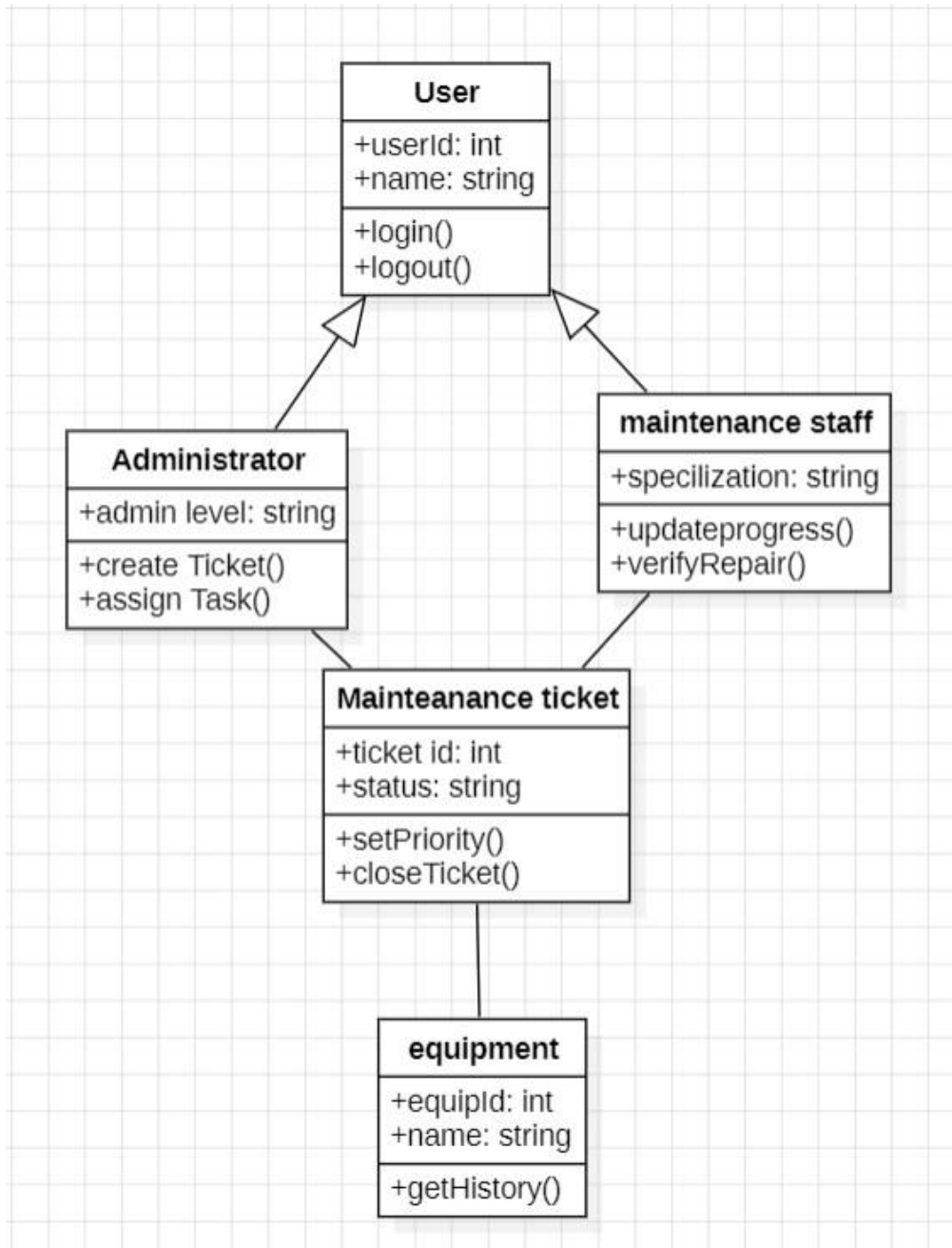


Logic: The Equipment class tracks the status (Working/Needs Repair) and purchase date of each asset.

Relationships:

- **Aggregation:** The Inventory class serves as a container for all gym equipments in the gym.
- **Composition:** Each piece of Equipment is linked to multiple Maintenance Logs to track detailed service history like repair costs and descriptions

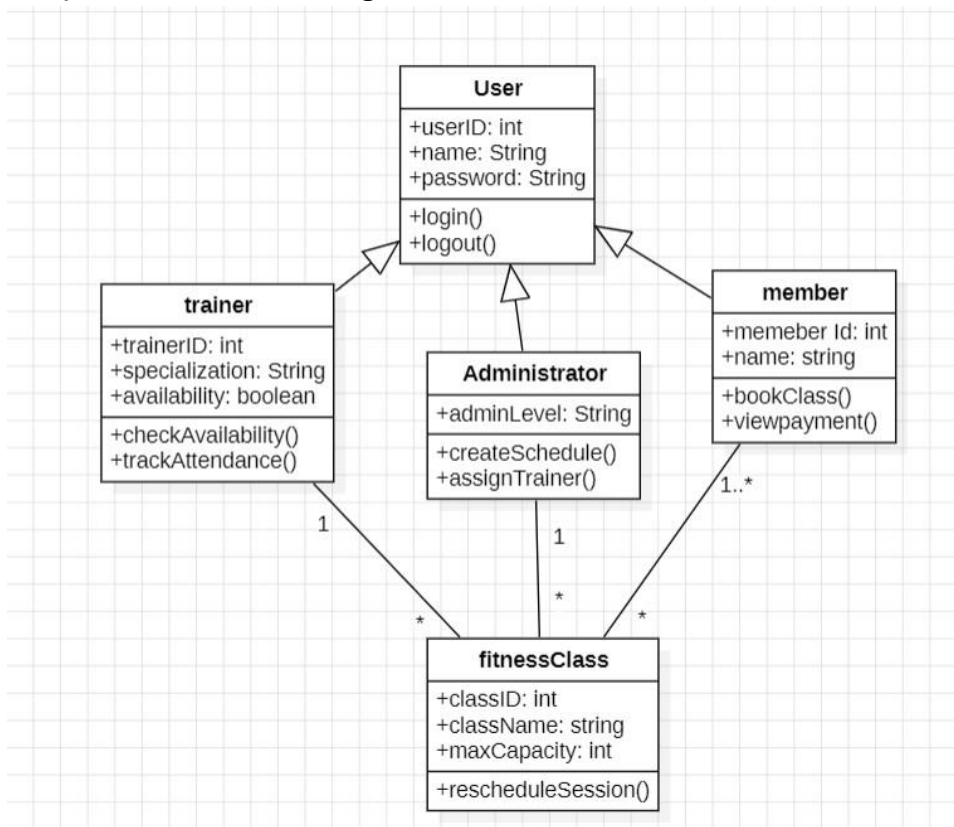
c) Maintenance Scheduling & Ticketing



Logic: The Maintenance Ticket acts as the central object, capturing the current status and priority of a repair task in the gym for all the equipment. **Relationships:**

- **Inheritance:** The Maintenance Staff and Administrator inherit from the User class. At the end everyone is uses the system , so they inherit from user .
- **Assignment:** The Administrator assigns tasks to Maintenance Staff through the Maintenance Ticket.
- **Asset Link:** Each ticket is associated with a specific piece of Equipment to identify the machine needing work.

d) Trainer & Class Management

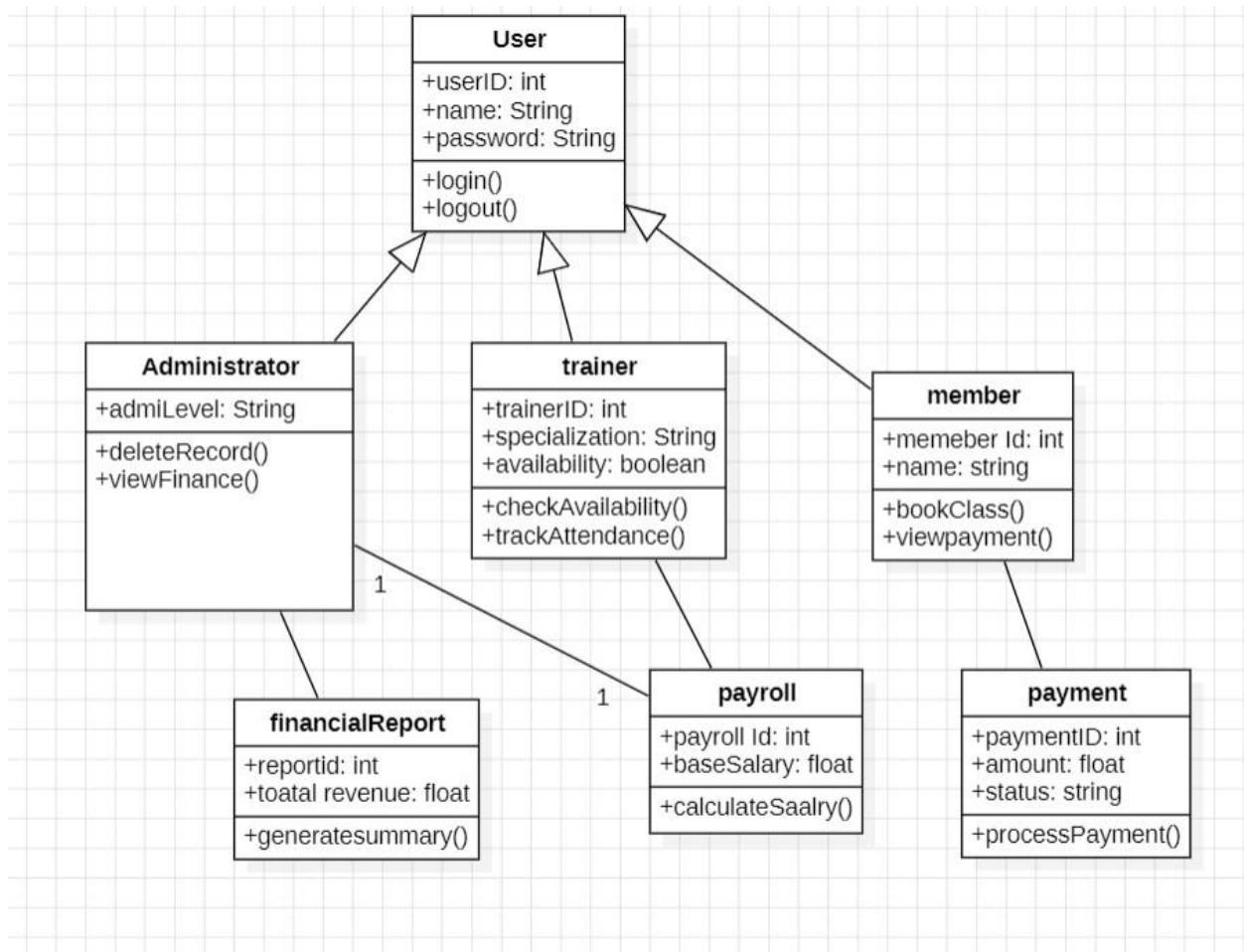


Logic: It ensures that a Trainer has their availability checked before being assigned to a FitnessClass.

Relationships:

- **Schedule:** One Trainer can lead multiple FitnessClasses
- **Booking:** Members interact with the FitnessClass through the bookClass() method, with multiple members able to join a single class
- **Inheritance:** The Maintenance Staff and Administrator inherit from the User class. At the end everyone is uses the system , so they inherit from user .

e) Billing & Analytics , Staff Payroll/Payments



Logic: It automates member fee collection and calculates staff salaries through the Payroll class.

Relationships:

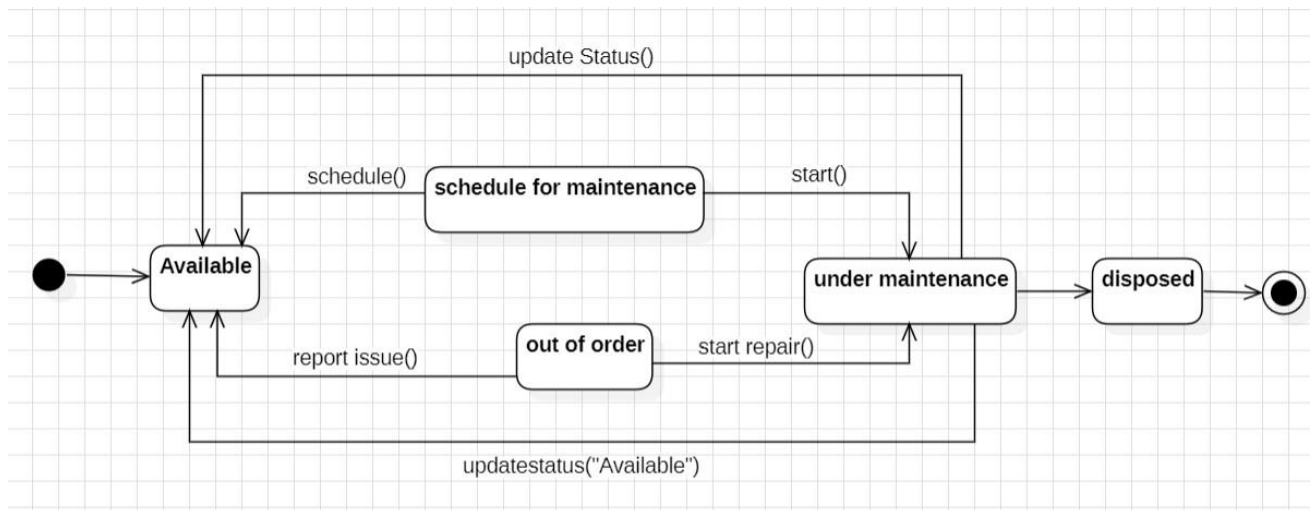
- **Revenue:** A Member is associated with Payment records for tracking dues .
- **Expense:** The Administrator manages Payroll for all staff members and generates a Financial Report to view total revenue
- **Inheritance:** The Maintenance Staff and Administrator inherit from the User class. At the end everyone is uses the system , so they inherit from user .

- State chart diagrams

The state chart diagrams are taken from equipment and asset tracking module and this module consists of 3 classes

- 1. Equipment State chart
- 2. Maintenance Log State chart
- 3. Inventory State chart

Equipment State chart

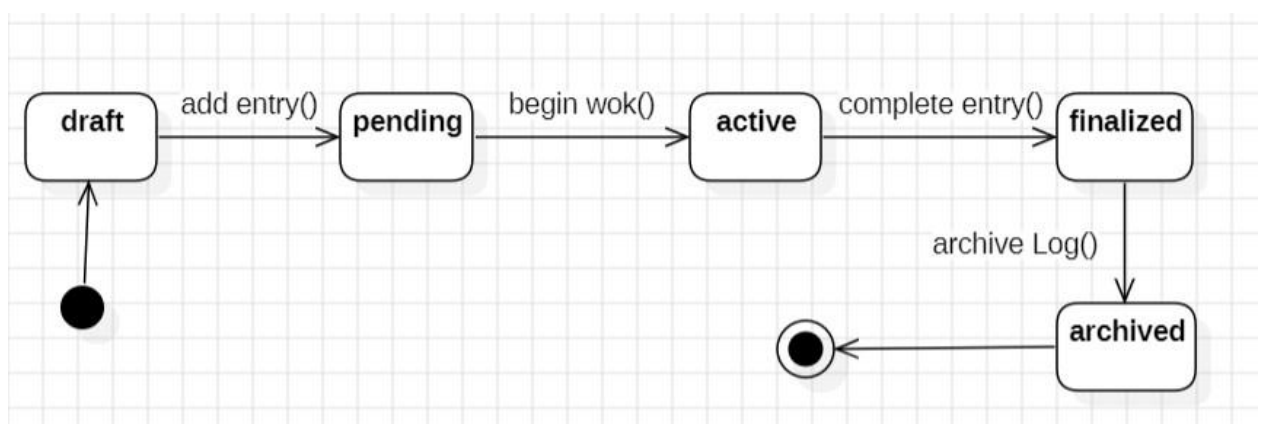


This state chart shows how equipment class works

The module starts and

If the equipment is available , schedule maintenance , start maintenance , and dispose and it ends

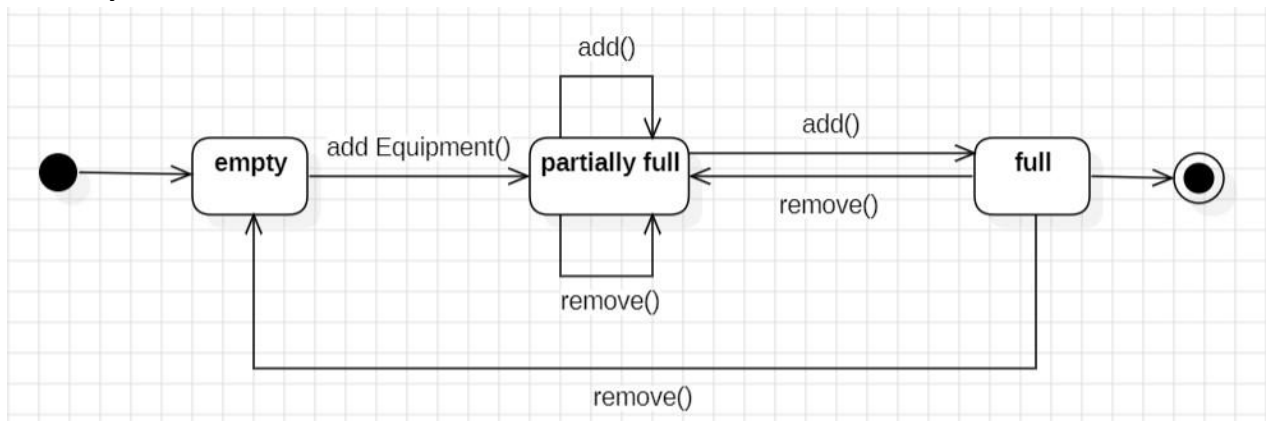
Maintenance Log State chart



This state chart shows how the maintenance log works

The module starts and drafts and enter maintenance , see pending and activate them and complete them , finalize and end

Inventory State chart

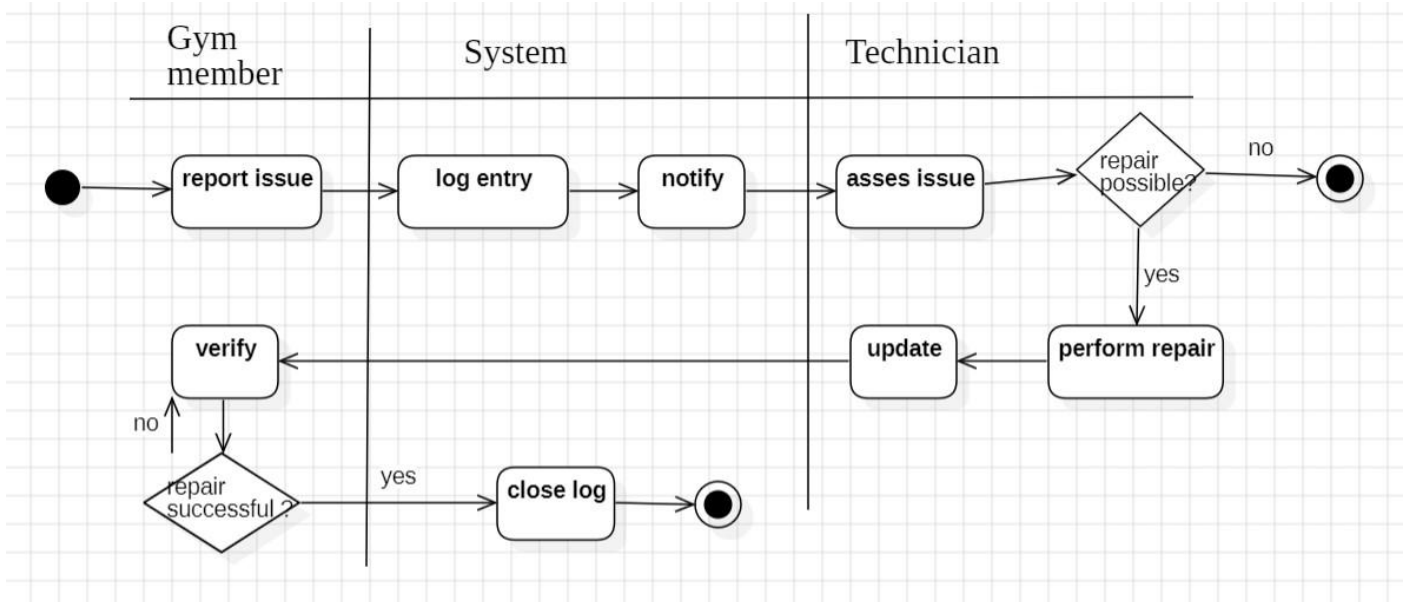


This state chart shows how the inventory works

Once we start we check if empty we add equipment , after it gets partially filled we can either add or remove , after full also same

- Activity diagrams

Equipment & Asset Tracking

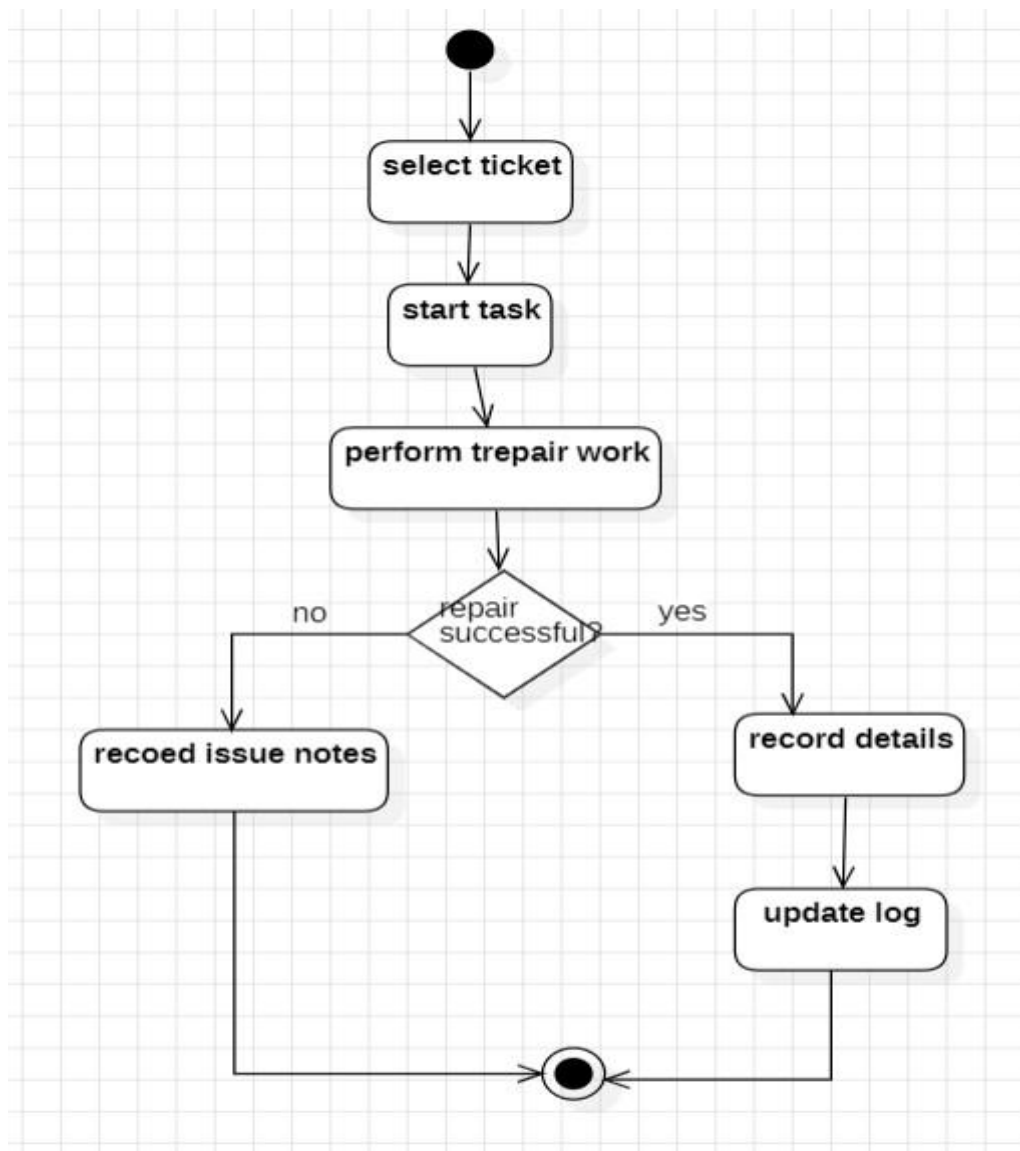


this activity diagram shows how an equipment issue is resolved from starting when it gets reported to how the report gets solved .

This diagram shows the end-to-end workflow when a piece of gym equipment breaks down. It uses **Swim lanes** to clearly define the responsibilities of the staff, the system, and the technician.

- **Trigger:** A staff member or gym member notices a problem and reports the equipment issue.
- **System Automation:** The system automatically creates a new **Maintenance Log** entry and sends a notification to the assigned technician.
- **Technician Assessment:** The technician inspects the machine. If the repair is impossible (e.g., the frame is cracked), the process ends with the equipment being retired.
- **Repair Execution:** If fixable, the technician performs the work and updates the log with specific details and the final cost.
- **Verification:** The staff member verifies the machine is working. If it's still faulty, it goes back to the technician; otherwise, the log is closed.

Maintenance Scheduling & Ticketing

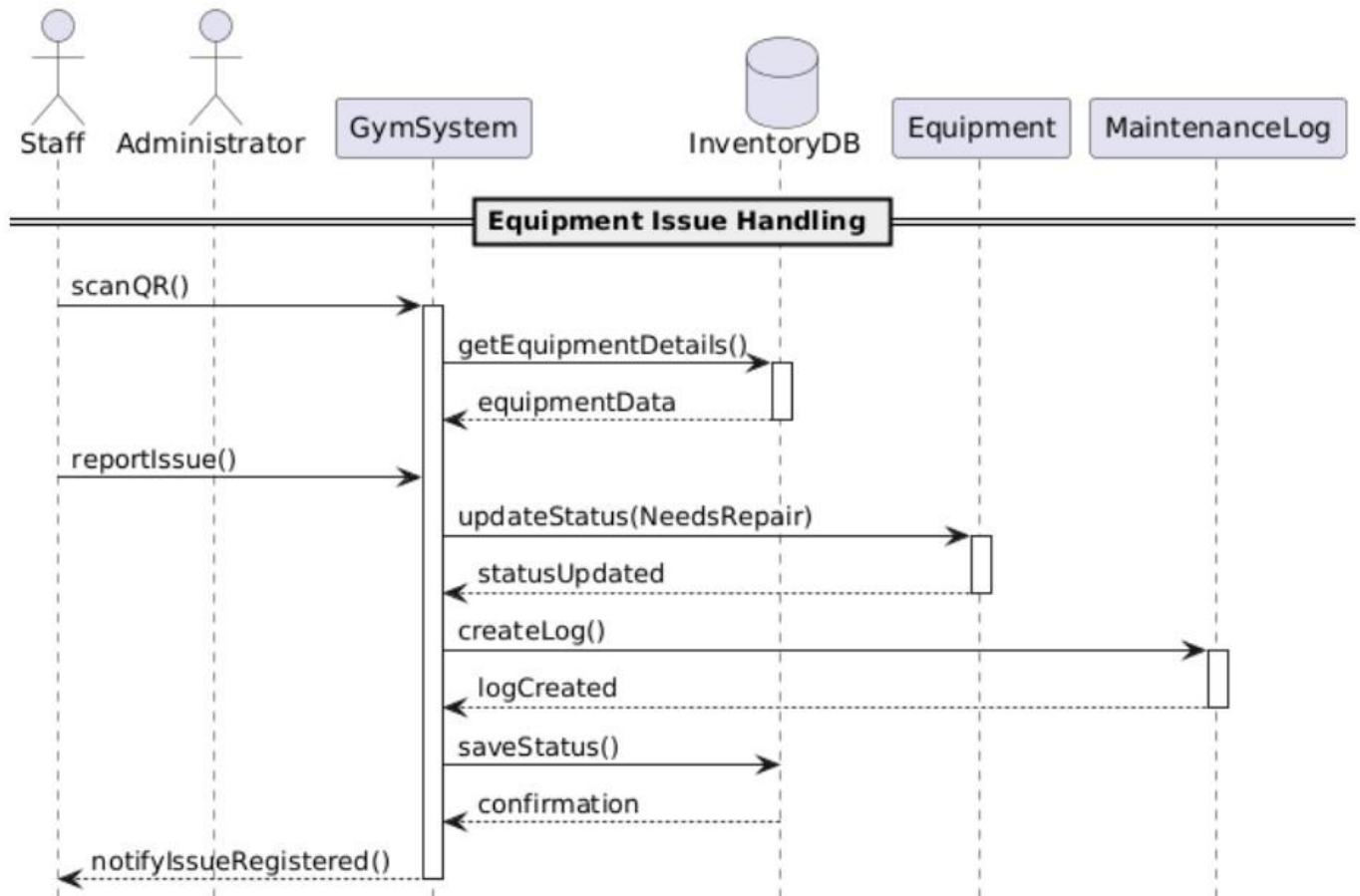


This diagram focuses specifically on the internal workflow of the **Maintenance Staff** user role from your second class diagram. It highlights the logical steps a technician takes within the app.

- **Task Selection:** The technician logs in and selects a specific ticket from their "Assigned" list.
- **Decision Logic:** After the physical work is done, the technician evaluates if the repair was successful.
- **Success Path:** If fixed, they record the final details and mark the ticket as "**Completed**", which triggers the `closeTicket()` method in the system.
- **Failure Path:** If the issue persists or needs a specific part, they record notes about the remaining issues, leaving the ticket open for further action.

- Sequence diagrams

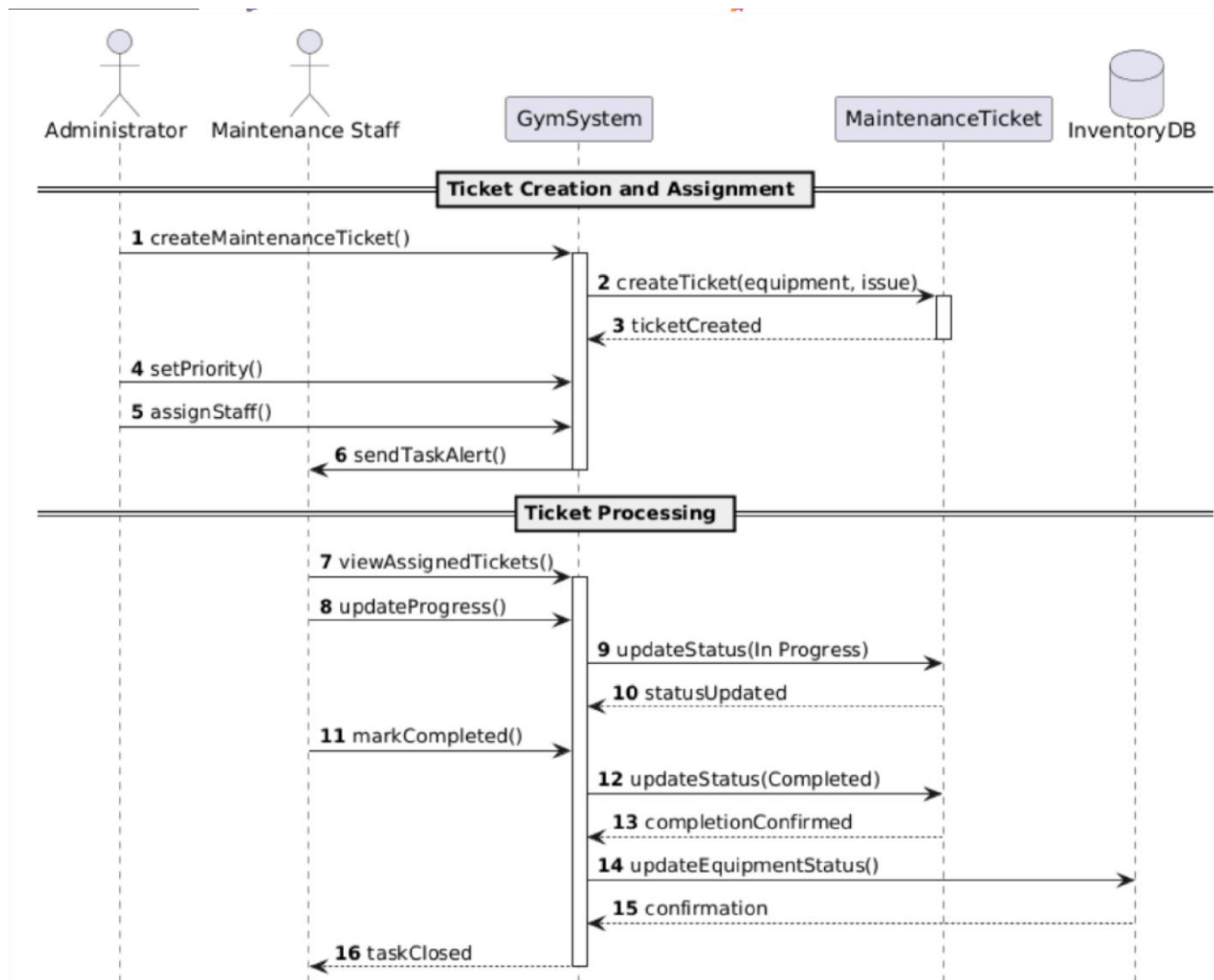
Equipment & Asset Tracking



This sequence diagram shows the interaction between the Staff, Administrator, and the Gym System during the process of reporting and managing equipment issues. The workflow begins when a staff member scans the equipment QR and reports a problem. The system retrieves equipment details from the Inventory Database, updates the equipment status to *Needs Repair*, and creates a Maintenance Log entry. After saving the updated information, the system sends a confirmation notification to the staff.

The diagram also represents the report generation process, where the Administrator requests the equipment health report. The system fetches the current equipment status from the database and generates the health report.

Overall, this diagram illustrates the step-by-step communication between system components involved in equipment monitoring, issue handling, maintenance recording, and health report generation.

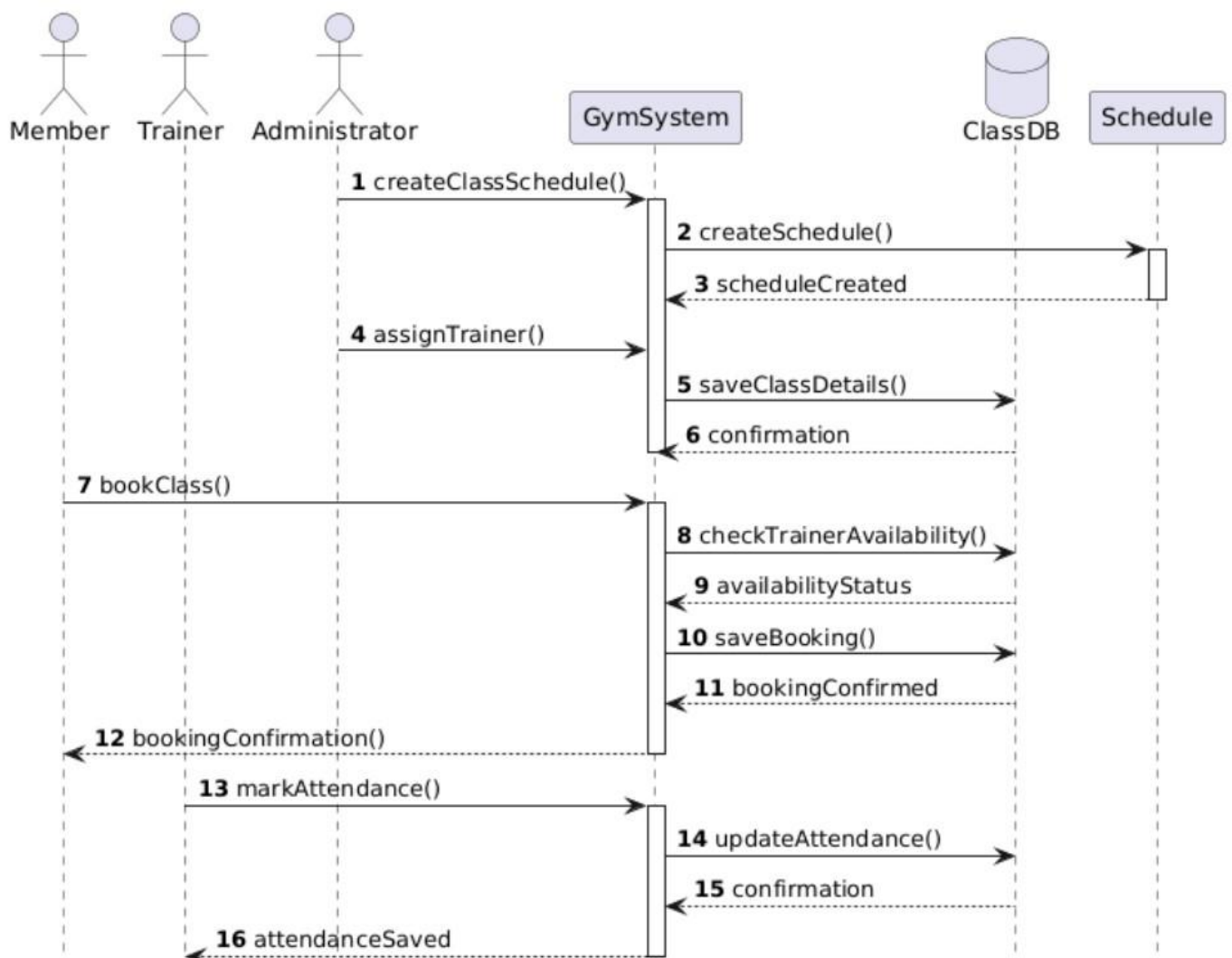


Maintenance Scheduling & Ticketing

This sequence diagram illustrates the interaction between the Administrator, Maintenance Staff, and the Gym System for handling maintenance tasks. The process begins when the Administrator creates a maintenance ticket for a faulty equipment, sets the priority level, and assigns the task to a maintenance staff member. The system then sends a task notification to the assigned staff. The maintenance staff views the assigned tickets, performs the repair work, and updates the task progress in the system. Once the repair is completed, the ticket status is marked as *Completed*, and the system updates the equipment status in the database.

This diagram represents the complete workflow of maintenance task creation, assignment, execution, and closure within the system.

Trainer & Class Management

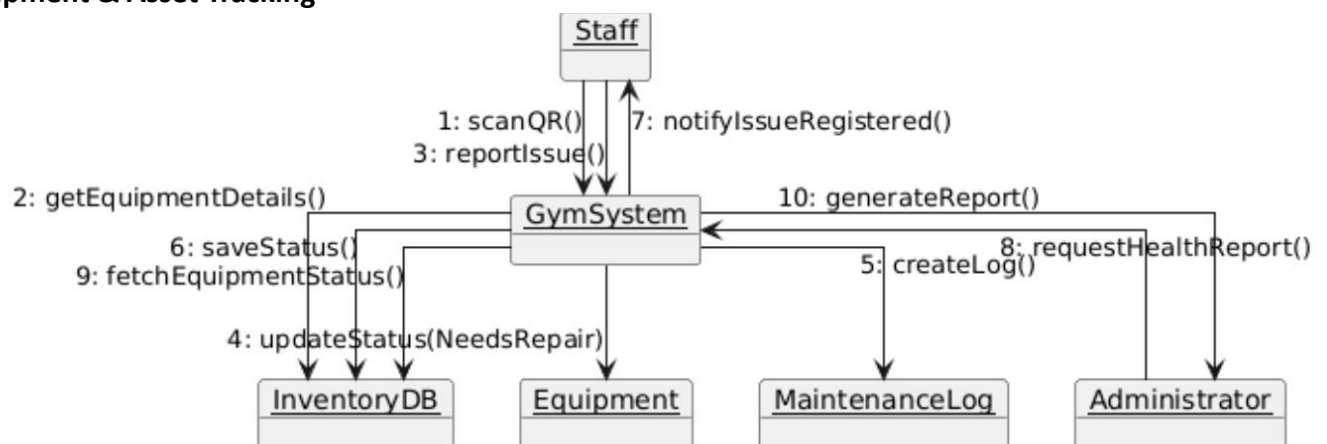


This sequence diagram shows the interaction between the Administrator, Member, Trainer, and the Gym System for managing fitness classes. The process begins when the administrator creates a class schedule and assigns a trainer. The system stores the class details in the database. The diagram also represents the class booking process, where a member requests to book a class. The system checks trainer availability, saves the booking, and sends a confirmation to the member. Additionally, after the session, the trainer marks attendance, and the system updates the attendance records.

This diagram illustrates the complete workflow of class scheduling, trainer assignment, member booking, and attendance management within the system.

- Collaboration diagrams

Equipment & Asset Tracking

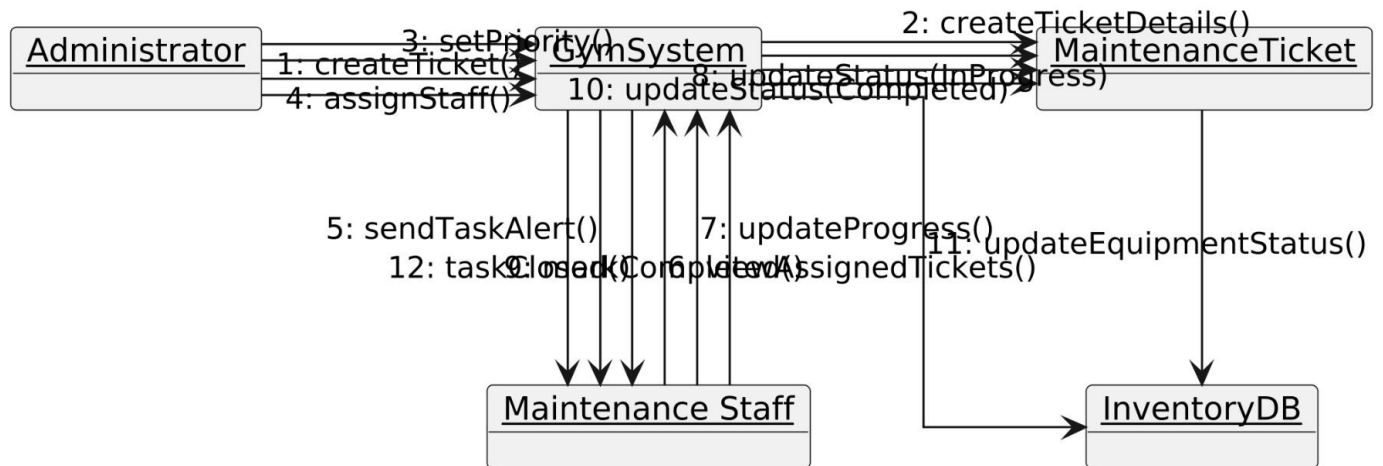


This collaboration diagram illustrates the communication between different system objects involved in monitoring and managing gym equipment. When a staff member scans the equipment QR and reports an issue, the Gym System interacts with the Inventory Database to retrieve equipment details, updates the equipment status, and creates a Maintenance Log entry. The updated status is then saved in the database, and a confirmation is sent to the staff.

The diagram also shows the reporting interaction, where the Administrator requests an equipment health report. The Gym System collects the current equipment status from the database and generates the report.

This diagram represents the message flow and coordination between system components responsible for equipment issue handling, maintenance tracking, and asset health monitoring.

Maintenance Scheduling & Ticketing

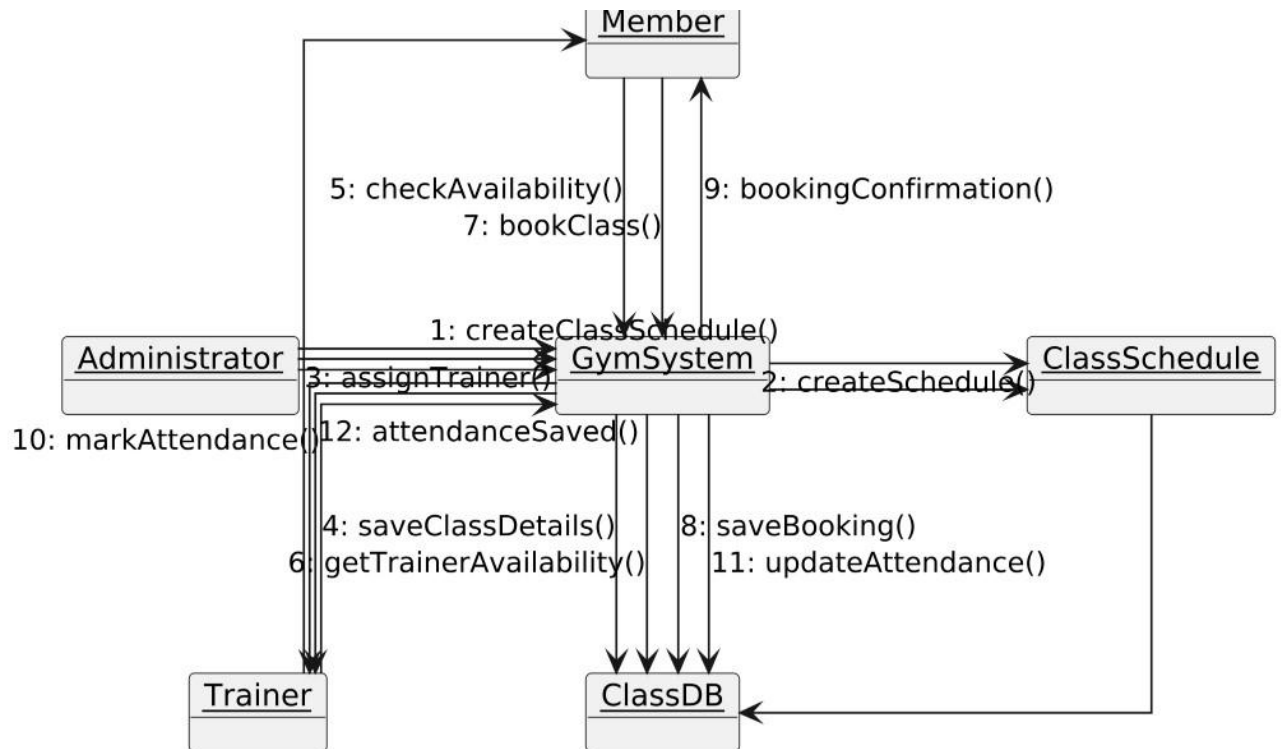


This collaboration diagram shows the communication between the Administrator, Maintenance Staff, and system components involved in managing maintenance tasks. The process begins when the Administrator creates a maintenance ticket, sets the priority level, and assigns the task to a maintenance staff member. The Gym System records the ticket details and sends a task notification to the assigned staff.

The maintenance staff then views the assigned tickets, performs the repair work, and updates the task progress. After the repair is completed, the system updates the ticket status, records the equipment status in the database, and sends a confirmation that the task has been closed.

This diagram represents the interaction and message flow between system objects responsible for maintenance task creation, assignment, execution, and completion.

Trainer & Class Management



This collaboration diagram illustrates the communication between the Administrator, Member, Trainer, and system components involved in managing fitness classes. The process begins when the Administrator creates a class schedule and assigns a trainer. The Gym System stores the class and schedule details in the database.

Members interact with the system to check trainer availability and book a class. The system verifies availability, records the booking, and sends a confirmation to the member. After the session, the Trainer updates the attendance, and the system saves the attendance details in the database.

This diagram represents the interaction and message flow between system objects responsible for class scheduling, trainer assignment, member booking, and attendance management.

ER diagram :

Entities :

Member
Trainer
Payment
Attendance

Member :

Member_id(PK)
Name
Phone
Membership_type

Trainer :

Trainer_id(PK)
Name
Specialization

Payment :

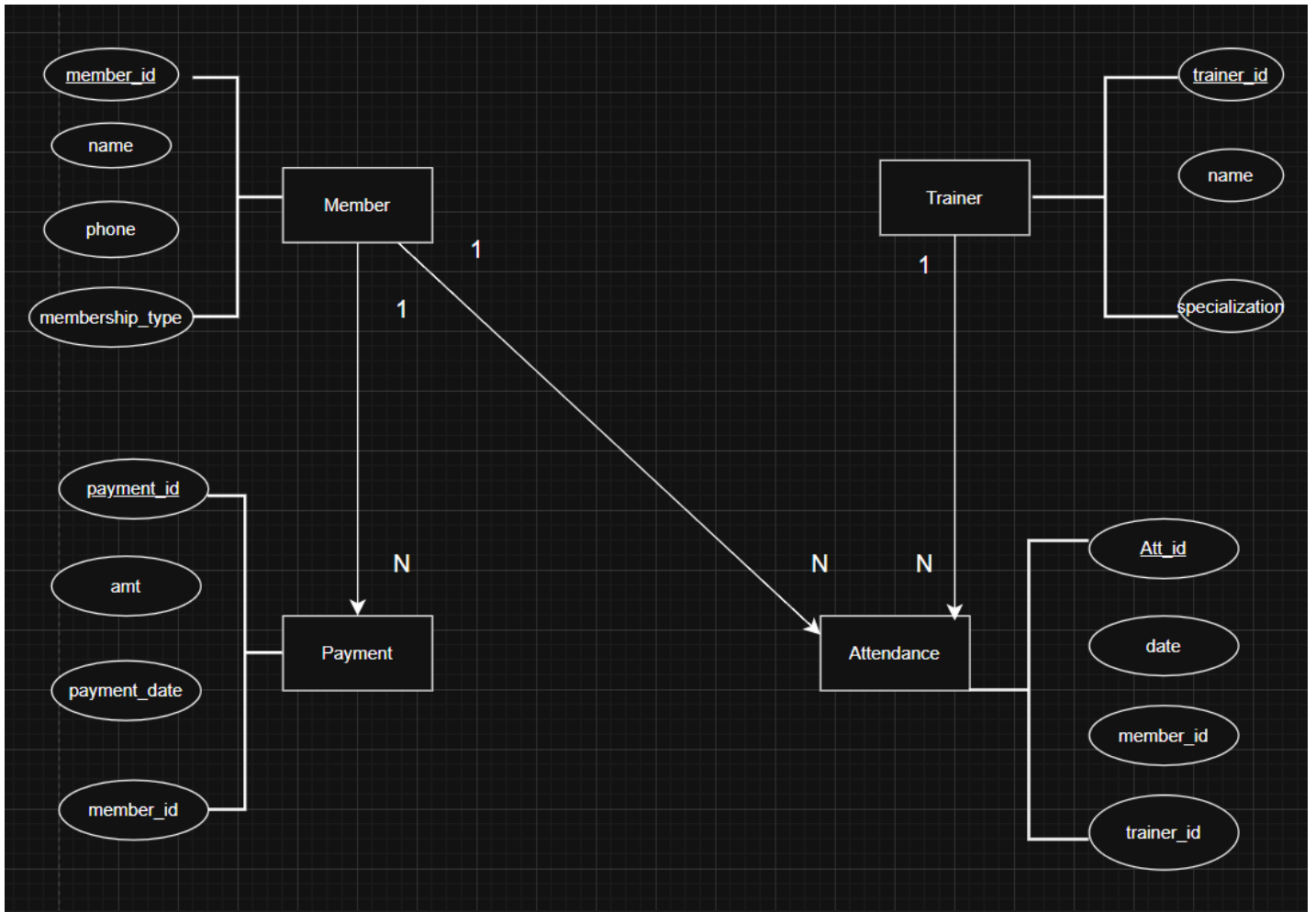
Payment_id(PK)
Amount
Payment_date
Member_id(FK)

Attendance :

Attendance_id(PK)
Date
Member_id(PK)
Trainer_id(PK)

Relationships :

Member (1)--->(N) payment
Member(1)--->(N) Attendance
Trainer(1)--->(N)Attendance



Dataflow Diagrams :

The dataflow diagram of the gym maintenance system consists of 2 levels:
DFD level 0 and DFD level 1.

DFD level 0:

External Entities :

Administrator

Member

Trainer

Process :

Gym Maintenance System

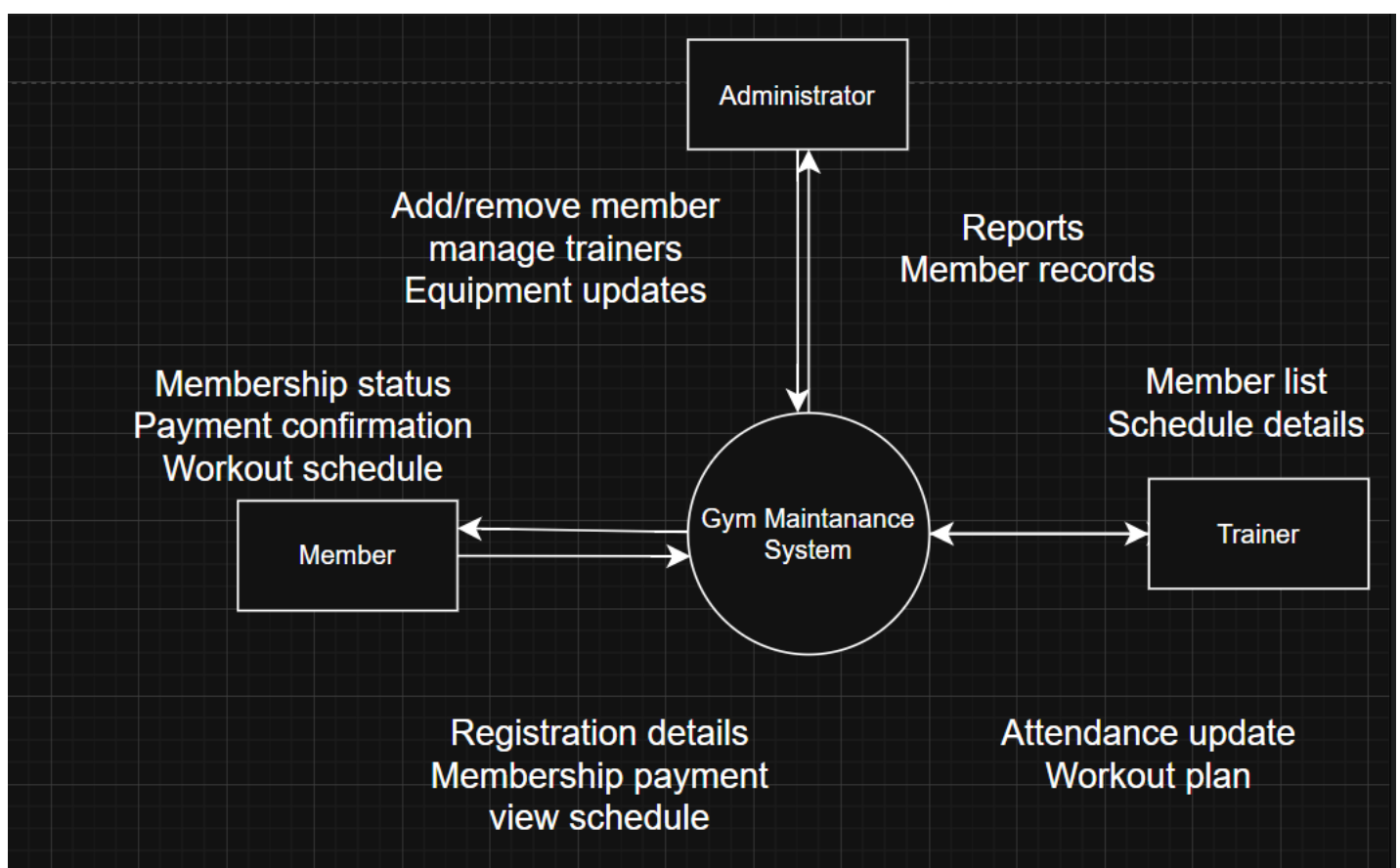
Dataflows :

System to Member---> Membership status, Payment confirmation, Workout schedule.

Member to system--->Registration details, Membership Payment, view schedule.

System to Administrator---> Reports, Member records.

Administrator to system--->add/remove members, manage trainers, and equipment updates.



DFD level 1:

Internal entities :

Member
Trainer
Administrator

Internal Processes :

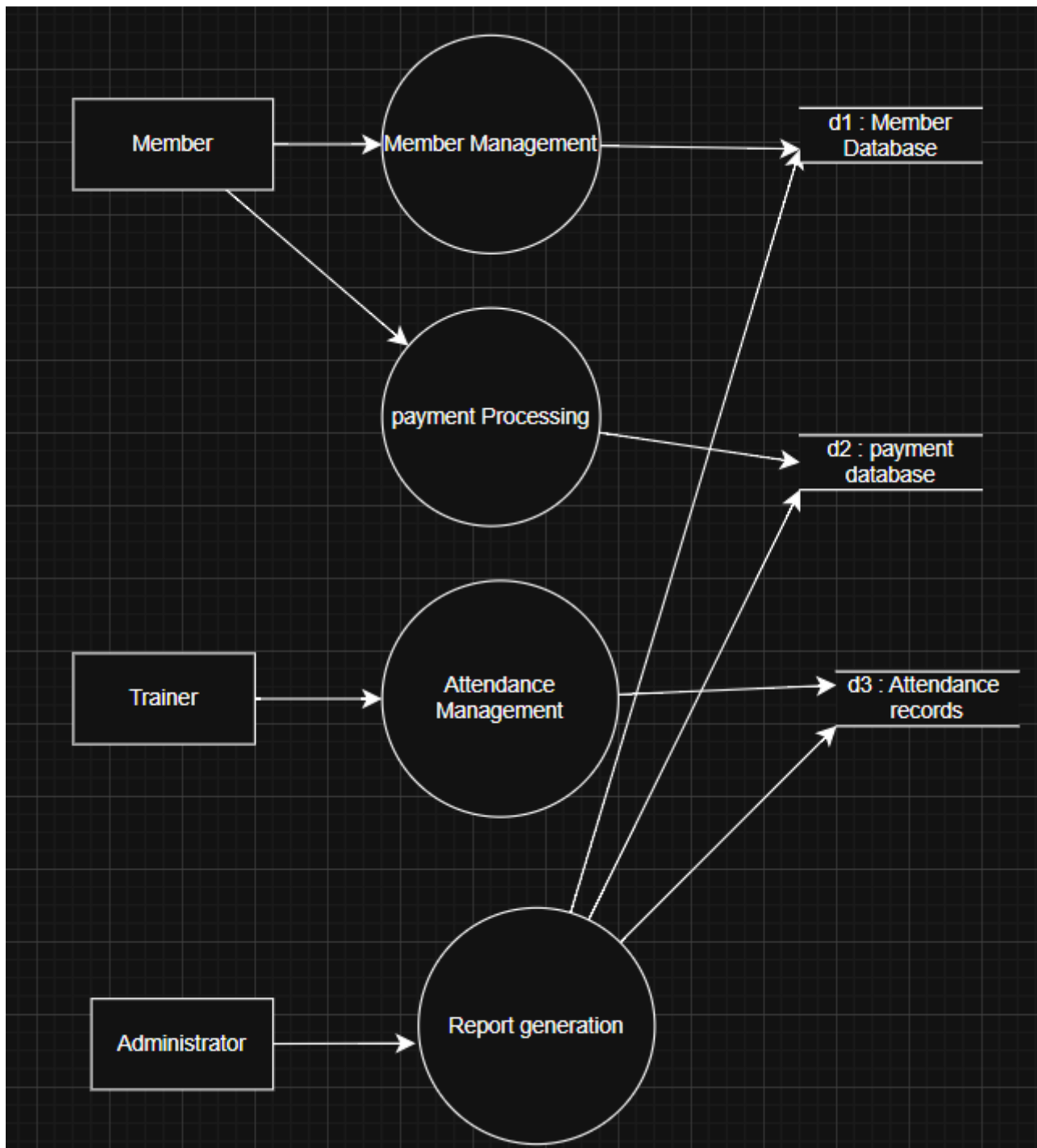
Member management
Payment processing
Attendance management
Report generation

Data stores :

Member database
Payment Database
Attendance Records

Dataflows :

Member to Member management, payment processing.
Member management to the member database.
Payment processing to the payment database.
Trainer to attendance management .
Attendance management to attendance records.
Administrator to Report generation.
Report generation to all the data stores.



Chapter-4 External Interface Requirements/Screenshots

4.1 User Interfaces

Gym Member Interfaces :

It allows the member to view their membership details, workout schedules, update their personal information (height, weight), and also view their payments.

Trainer Interface :

This interface is basically for the personal trainers; it gives access to the trainer about their trainee to monitor their workout progress and plans.

Administrator Interface :

It manages the count of members (registrations), details of the trainer, count of equipments and system reports. It can also modify the roles of users and look after the entire system.

4.2 Hardware Interfaces

- Usage of biometrics for attendance.
- Verifying memberships using barcode scanners or RFID.
- Mobile or desktop for accessing the application.
- Printing out receipts or reports by the printers.

4.3 Software Interfaces

Operating systems :

This system can run on the available os like Windows, macOS, or mobile operating systems, depending on the deployment.

Database Management System :

It stores all the details related to the member, the trainer, and also about the equipment(MySQL, MongoDB).

Payment Gateway :

It allows members to renew their membership securely.

APIs:

APIs are used for sending messages on their updates or payments via SMS or email.

4.4 Communication Interfaces

- It is done through HTTPS/HTTP.

- REST APIs are used for the communication between frontend and backend components.
- Internet, LAN , WiFi is needed for the system to function.

Chapter 5: Other Non-Functional Requirements

5.1 Performance Requirements :

- The system should always be ready to respond to the user's actions.
- It should handle multiple users simultaneously.
- Data updates and back-ups should be processed smoothly.

5.2 Safety Requirements

- Regular backups should be implemented.
- Recovery should be available.
- The system should handle errors smoothly.

5.3 Security Requirements

- The system can only be accessed by authorised users.
- User authentication is required before logging in.
- Data must be protected.

5.4 Software Quality Attributes

Reliability :

The system must be operated consistently.

Usability :

The interface must be simple and easy to understand.

Scalability :

The system must support expansion in the future.

Maintainability :

The system should allow updates and notifications.

5.5 Business Policies/Rules

- Members must access the gym services.
- The membership fee must be paid before renewal.

Chapter 6 – Testing

Sample code 1

Code for maintenance control unit

```
# Maintenance Control Unit
```

```
import unittest
```

```
def maintenance_control(equipment_status, priority):
```

```
    if equipment_status not in ["Working", "Needs Repair"]:
        return "Invalid Status"
```

```
    if equipment_status == "Working":
        return "No Maintenance Required"
```

```
    if priority not in ["Low", "Medium", "High"]:
        return "Invalid Priority"
```

```
    if priority == "High":
        return "Immediate Maintenance"
```

```
    elif priority == "Medium":
        return "Schedule Soon"
```

```
    else:
        return "Normal Maintenance"
```

WHITE BOX

```
class TestMaintenanceControl(unittest.TestCase):
```

```
    # White Box Paths
```

```
    def test_invalid_status(self):
        self.assertEqual(
            maintenance_control("Broken", "High"),
            "Invalid Status"
        )
```



```

def test_working_status(self):
    self.assertEqual(
        maintenance_control("Working", "Low"),
        "No Maintenance Required"
    )

def test_invalid_priority(self):
    self.assertEqual(
        maintenance_control("Needs Repair", "Critical"),
        "Invalid Priority"
    )

def test_high_priority(self):
    self.assertEqual(
        maintenance_control("Needs Repair", "High"),
        "Immediate Maintenance"
    )

def test_medium_priority(self):
    self.assertEqual(
        maintenance_control("Needs Repair", "Medium"),
        "Schedule Soon"
    )

def test_low_priority(self):
    self.assertEqual(
        maintenance_control("Needs Repair", "Low"),
        "Normal Maintenance"
    )

```

```

# Run Tests
if __name__ == "__main__":
    unittest.main()

```

Black Box Testing

```

test_cases = [
    ("Broken", "High"),
    ("Working", "Low"),
    ("Needs Repair", "Critical"),
    ("Needs Repair", "High"),

```

```

    ("Needs Repair", "Medium"),
    ("Needs Repair", "Low")
]

print("Black Box Testing – Maintenance Control Unit\n")

for status, priority in test_cases:
    result = maintenance_control(status, priority)
    print(f"Input: ({status}, {priority}) -> Output: {result}")

```

OUTPUT

```

===== RESTART: C:/Users/KARTHIK/maint.py =====
.....
-----
Ran 6 tests in 0.074s

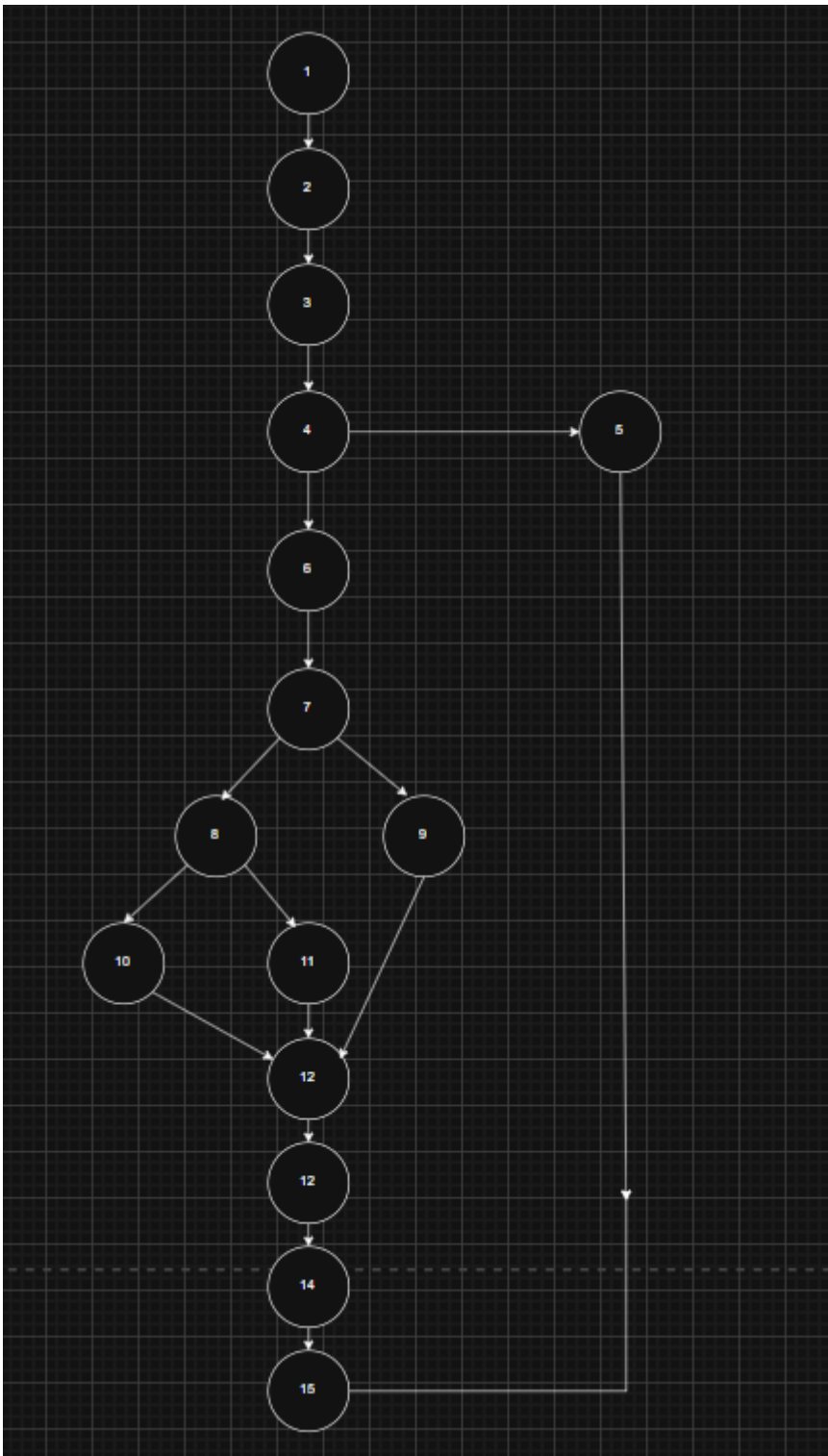
OK
===== RESTART: C:\Users\KARTHIK\Downloads\maintenance code.py =====
Black Box Testing - Maintenance Control Unit

Input: (Broken, High) -> Output: Invalid Status
Input: (Working, Low) -> Output: No Maintenance Required
Input: (Needs Repair, Critical) -> Output: Invalid Priority
Input: (Needs Repair, High) -> Output: Immediate Maintenance
Input: (Needs Repair, Medium) -> Output: Schedule Soon
Input: (Needs Repair, Low) -> Output: Normal Maintenance
|

```

WHITE BOX TESTING

Flow diagram



Node No Description

1	Start
2	Input equipment_status and priority
3	Validate equipment_status
4	Check if status = "Working"
5	Display "No Maintenance Required"
6	Validate priority
7	Check if priority = High
8	Check if priority = Medium
9	Schedule Immediate Maintenance
10	Schedule Maintenance Soon
11	Schedule Normal Maintenance
12	Update Maintenance Log
13	Notify Maintenance Staff
14	Confirm Maintenance Request
15	End

Cyclomatic Complexity:

$V(G) = \text{Predicate Nodes} + 1$

Predicate nodes:

1. status validation
2. status == Working
3. priority validation
4. priority == High
5. priority == Medium

$V(G) = 5 + 1 = 6$

So 6 independent paths are required.

Independent paths

Path No	Independent Path (Node Sequence)	Test Input (equipment status, priority)	Expected Output
P1	1,2,3,4,15	("Broken", "High")	Invalid Status
P2	1,2,3,5,15	("Working", "Low")	No Maintenance Required
P3	1,2,3,6,7,15	("Needs Repair", "Critical")	Invalid Priority
P4	1,2,3,6,8,9,15	("Needs Repair", "High")	Immediate Maintenance
P5	1,2,3,6,8,10,11,15	("Needs Repair", "Medium")	Schedule Soon
P6	1,2,3,6,8,10,12,13,15	("Needs Repair", "Low")	Normal Maintenance

BLACK BOX TESTING

Boundary Value Analysis

BB No	Test Input (equipment status, priority)	Expected Output	Technique
BB1	("Broken", "High")	Invalid Status	Invalid Status Class
BB2	("Working", "Low")	No Maintenance Required	Valid Status Class
BB3	("Needs Repair", "Critical")	Invalid Priority	Invalid Priority Class
BB4	("Needs Repair", "High")	Immediate Maintenance	High Priority Class
BB5	("Needs Repair", "Medium")	Schedule Soon	Medium Priority Class
BB6	("Needs Repair", "Low")	Normal Maintenance	Low Priority Class

Equivalence Partitioning

Class ID	Input Condition	Type
EC1	Status = Working	Valid
EC2	Status = Needs Repair	Valid
EC3	Status = any other value	Invalid
EC4	Priority = High	Valid
EC5	Priority = Medium	Valid
EC6	Priority = Low	Valid
EC7	Priority = any other value	Invalid

Test Case	equipment_status	priority	Expected Output	Class Covered
TC1	Working	Low	No Maintenance Required	EC1
TC2	Needs Repair	High	Immediate Maintenance	EC2, EC4
TC3	Needs Repair	Medium	Schedule Soon	EC2, EC5
TC4	Needs Repair	Low	Normal Maintenance	EC2, EC6
TC5	Broken	High	Invalid Status	EC3
TC6	Needs Repair	Critical	Invalid Priority	EC7

Sample code 2

Code for billing control unit

Billing Control Unit

```
import unittest
```

```
def billing_control(months, amount_per_month):
```

```
    if months <= 0:
        return "Invalid Months"
```

```
    if amount_per_month <= 0:
        return "Invalid Amount"
```

```
    total = months * amount_per_month
```

```
    if total >= 10000:
        return "Premium"
```

```
    elif total >= 5000:
        return "Gold"
```

```
    elif total >= 2000:
        return "Silver"
```

```
    else:
        return "Basic"
```

WHITE BOX

```
class TestBillingControl(unittest.TestCase):
```

```
    # White Box Independent Paths
```

```
    def test_invalid_months(self):
        self.assertEqual(
            billing_control(-1, 1000),
            "Invalid Months"
        )
```

```
    def test_invalid_amount(self):
        self.assertEqual(
```

```

        billing_control(5, -100),
        "Invalid Amount"
    )

def test_premium(self):
    self.assertEqual(
        billing_control(12, 1000),
        "Premium"
    )

def test_gold(self):
    self.assertEqual(
        billing_control(6, 1000),
        "Gold"
    )

def test_silver(self):
    self.assertEqual(
        billing_control(3, 1000),
        "Silver"
    )

def test_basic(self):
    self.assertEqual(
        billing_control(1, 1000),
        "Basic"
    )

# Run Tests
if __name__ == "__main__":
    unittest.main()

# Black Box Testing

test_cases = [
    (-1, 1000),
    (1, -100),
    (1, 1000),
    (3, 1000),
    (6, 1000),
    (12, 1000)
]

print("Black Box Testing – Billing Control Unit\n")

```



```
for months, amount in test_cases:
    result = billing_control(months, amount)
    print(f"Input: ({months}, {amount}) -> Output: {result}")
```

OUTPUT

```
===== RESTART: C:/Users/KARTHIK/maint.py =====
.....
```

```
-----
Ran 6 tests in 0.017s
```

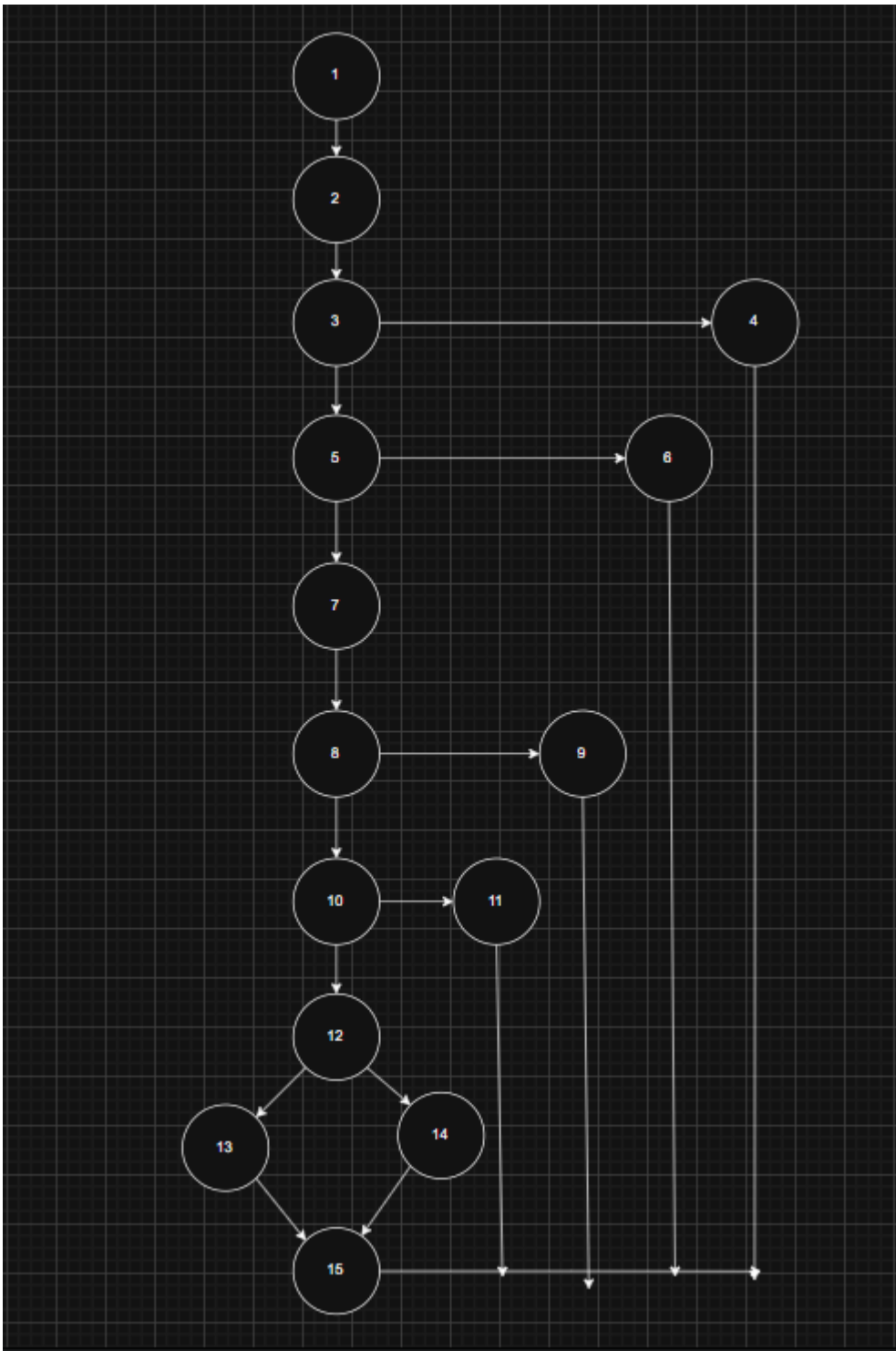
```
OK
```

```
===== RESTART: C:\Users\KARTHIK\Downloads\maintenance code.py =====
Black Box Testing - Billing Control Unit
```

```
Input: (-1, 1000) -> Output: Invalid Months
Input: (1, -100) -> Output: Invalid Amount
Input: (1, 1000) -> Output: Basic
Input: (3, 1000) -> Output: Silver
Input: (6, 1000) -> Output: Gold
Input: (12, 1000) -> Output: Premium
|
```

WHITE BOX TESTING

FLOW DIAGRAM



Node No	Description
1	Start
2	Input months and amount per month
3	Check if months ≤ 0
4	Display "Invalid Months"
5	Check if amount per month ≤ 0
6	Display "Invalid Amount"
7	Calculate total = months $\times$ amount per month
8	Check if total ≥ 10000
9	Assign Membership = Premium
10	Check if total ≥ 5000
11	Assign Membership = Gold
12	Check if total ≥ 2000
13	Assign Membership = Silver
14	Assign Membership = Basic
15	End

Cyclomatic Complexity:

$$V(G) = \text{Predicate Nodes} + 1$$

Predicate nodes:

months ≤ 0 (months validation)

amount per month ≤ 0 (amount validation)

total ≥ 10000 (Premium decision)

total ≥ 5000 (Gold decision)

total ≥ 2000 (Silver decision)

$$V(G) = 5 + 1 = 6$$

So 6 independent paths are required.

INDEPENDENT PATHS

Path No	Independent Path (Node Sequence)	Test Input (bill amount)	Expected Output
P1	1,2,3,4,12	-50	Invalid Bill Amount
P2	1,2,3,5,6,12	0	No Payment Required
P3	1,2,3,5,7,8,12	500	Bill Generated – Standard Payment
P4	1,2,3,5,7,9,10,12	3000	Bill Generated – Discount Applied
P5	1,2,3,5,7,9,11,12	8000	Bill Generated – Premium Billing

Path No	Independent Path (Node Sequence)	Test Input (bill amount)	Expected Output
P6	1,2,3,5,7,9,10,7,9,11,12	3500, 8200	Multiple Billing Operations (Loop Executes Multiple Times)

BLACK BOX TESTING

Boundary Value Analysis

BB No	Test Input (months, amount per month)	Expected Output	Technique
BB1	(-1, 1000)	Invalid Months	Below Boundary
BB2	(1, -100)	Invalid Amount	Below Boundary
BB3	(1, 1000)	Basic	Normal Case
BB4	(3, 1000)	Silver	Boundary Range
BB5	(6, 1000)	Gold	Boundary Range
BB6	(12, 1000)	Premium	Upper Range

Equivalence Partitioning

Class ID	Input Condition	Type
EC1	months ≤ 0	Invalid
EC2	months > 0	Valid
EC3	Amount per month ≤ 0	Invalid
EC4	Amount per month > 0	Valid
EC5	total ≥ 10000	Premium
EC6	$5000 \leq \text{total} < 10000$	Gold
EC7	$2000 \leq \text{total} < 5000$	Silver
EC8	total < 2000	Basic

Test Case	months	amount	Expected Output	Class Covered
TC1	-1	1000	Invalid Months	EC1
TC2	3	-500	Invalid Amount	EC3
TC3	12	1000	Premium	EC5
TC4	6	1000	Gold	EC6
TC5	3	1000	Silver	EC7
TC6	1	1000	Basic	EC8

Chapter 7 – Conclusion

The Software Requirements Specification (SRS) document for the Gym Management System provides a comprehensive description of the system's functional and non-functional requirements. The document outlines the major modules of the system, including Equipment & Asset Tracking, Maintenance Scheduling & Ticketing, Billing & Analytics, Staff Payroll & Payments, and Trainer & Class Management.

Through the use of UML diagrams such as sequence diagrams, collaboration diagrams, and flow graphs, the interactions between system components and users were clearly represented.

Additionally, both white box testing and black box testing techniques were applied to ensure that the system logic is well structured and testable. Concepts such as cyclomatic complexity, independent path testing, boundary value analysis, and equivalence partitioning were used to validate the system design.

Overall, this document establishes a clear foundation for the development and implementation of the Gym Management System, ensuring that the system will operate efficiently, maintain data accuracy, and support the operational needs of gym administrators, trainers, and staff.

Chapter 8 – References

1. UML Documentation – <https://www.uml.org>
2. Plant UML Documentation – <https://plantuml.com>
3. Draw.io Diagram Tool – <https://app.diagrams.net>
4. Professor's PPT Notes
5. STAR UML

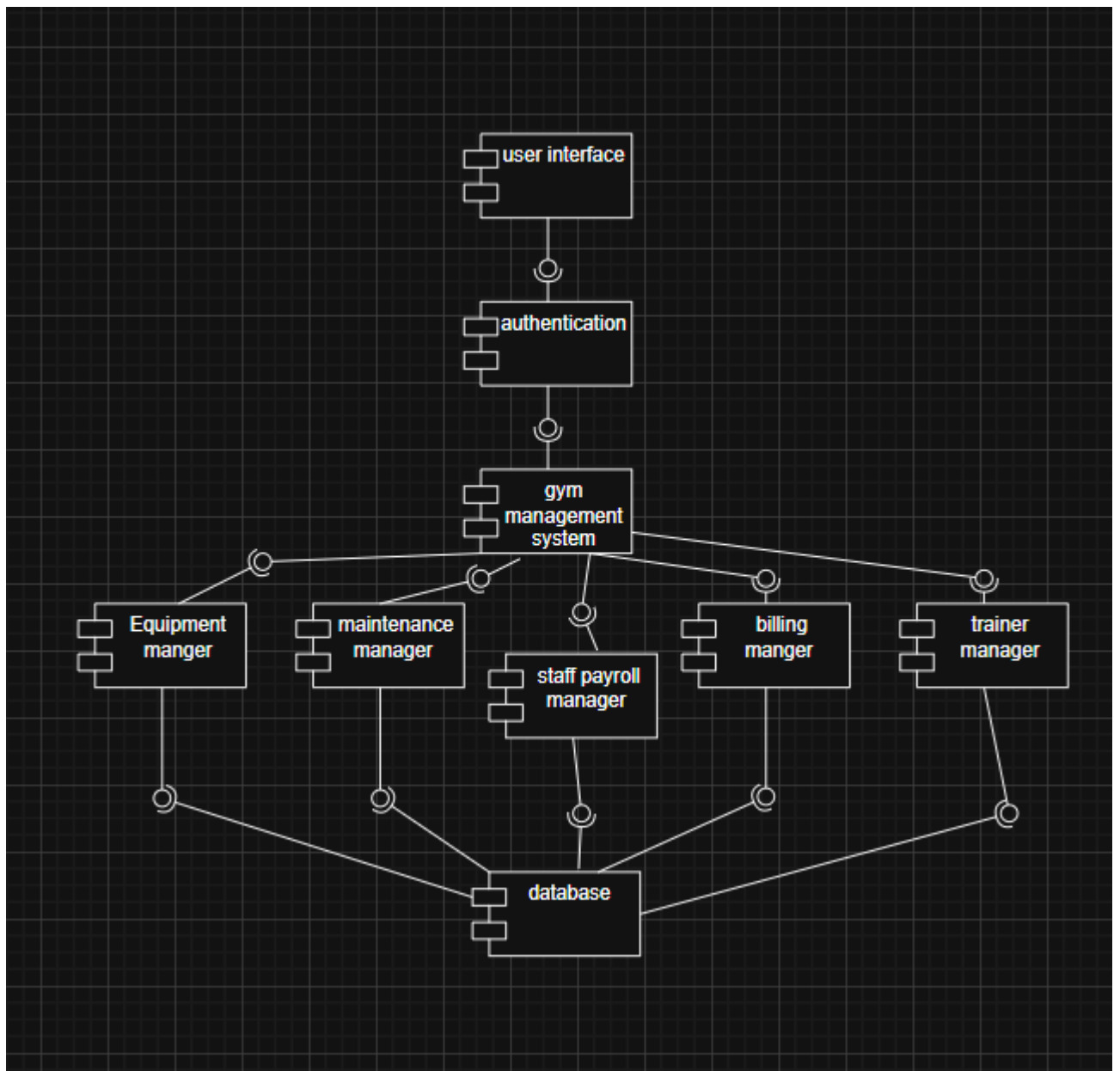
Chapter 9 – Acknowledgment

I would like to acknowledge that when I was preparing the document it gave the knowledge on how the software works internally, came to an understanding on how Star UML works.

I thank my faculty who has taught about the SRS document with great patience and my peers who helped me when I was stuck. Finally acknowledge the foundational topics and design topics that were the technical chapters of this document.

DESIGN ARCHITECTURE

COMPONENT-LEVEL DESIGN ELEMENTS:



Components Overview

We identified seven main components while designing this system. Here is what each one does and how it connects to the rest:

User Interface – This is the face of the system — the screens that the admin and staff actually see and use. It shows dashboards, member forms, equipment panels, and attendance records. Since it needs to know who is logged in before showing anything sensitive, it relies on Authentication. It also pulls live data from the Equipment Manager and Billing Manager to keep the screens up to date.

Authentication – Before anyone can do anything in the system, they go through this component. It checks login credentials, creates session tokens, and makes sure each user only sees what their role allows — whether they are an admin, a receptionist, or a maintenance worker. It reads and writes user data from the Database.

Equipment Manager – This component keeps track of every machine in the gym. It records whether a treadmill is working fine, needs a repair, or is currently under maintenance. It also stores purchase dates and service history. The moment a fault gets reported, it notifies the Maintenance Manager to create a repair ticket.

Maintenance Manager – Think of this as the repair desk of the system. When something breaks, this component creates a ticket, marks how urgent the fix is, and assigns the job to the right maintenance staff member. It keeps updating the ticket as work progresses and finally closes it once the repair is confirmed.

Billing Manager – This component handles everything related to money coming in. It records fee payments, flags members whose dues are overdue, sends out reminders when memberships are about to expire, and generates invoices. At the end of the month, it also produces a revenue summary for the admin.

Trainer Manager – Whenever a new fitness class needs to be scheduled, this component takes care of it. It checks whether a trainer is free, saves the booking, and records who attended each session. Members can also use it to book personal training slots.

Staff Payroll Manager – At the end of every month, this component calculates how many hours each staff member worked — whether they are a receptionist, a cleaner, a security guard, or a maintenance technician — and processes their salary accordingly. It also produces a payroll report for the admin to review.

Dependency Relationships

The way these components connect to each other follows a clear logic:

- The User Interface always checks in with Authentication first, before displaying any screen that contains member or financial data.
- It also reaches out to the Equipment Manager and Billing Manager to show real-time status on the dashboard.
- Whenever the Equipment Manager detects a fault, it immediately hands the job over to the Maintenance Manager to open a repair ticket.

- Almost every component — Authentication, Equipment Manager, Maintenance Manager, Billing Manager, Trainer Manager, and Payroll Manager — reads from and writes to the central Database, since that is where all the data actually lives.
- Both the Billing Manager and the Payroll Manager check with Authentication before recording any transaction, so that payments and salaries are always linked to the right account.

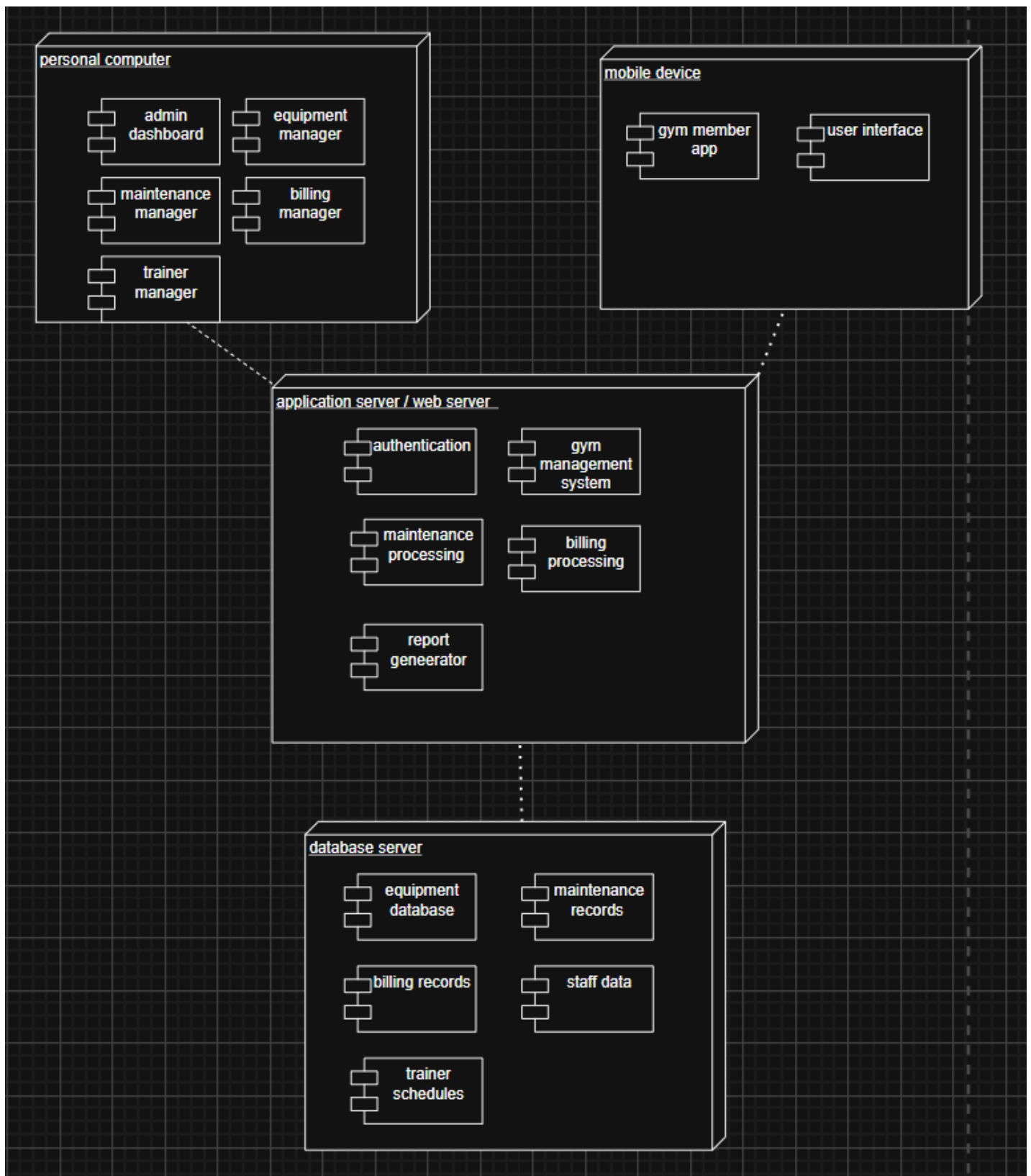
Component Diagram Description

The component diagram represents the high-level architecture of the **Gym Management System** and shows how different system modules interact with each other. The **User Interface** acts as the point of interaction where administrators or staff access the system. Before accessing the system features, users must go through the **Authentication** component, which verifies login credentials and ensures secure access.

After successful authentication, the request is handled by the **Gym Management System**, which acts as the central controller of the application. It coordinates different functional modules such as **Equipment Manager**, **Maintenance Manager**, **Billing Manager**, **Trainer Manager**, and **Staff Payroll Manager**. Each of these components is responsible for managing specific operations within the gym, such as tracking equipment, handling maintenance requests, managing billing and payments, scheduling trainers, and processing staff payroll.

All components interact with a common **Database**, which stores important information such as equipment details, maintenance records, payment data, trainer schedules, and staff information. This modular design helps in organizing system functionality efficiently and allows easier maintenance, scalability, and future enhancements of the system.

Deployment level design elements :



- **Personal Computer (Reception / Admin Desk):** This is the main workstation at the gym's front desk or admin office. It runs the desktop GUI that the staff use day-to-day. We kept the hardware requirements realistic — a standard Windows PC with an i3 processor and 4 GB of RAM is enough to run this smoothly. It sends requests to the Application Server over the gym's local network.
- **Mobile Device:** Gym members do not interact with the main desktop application. Instead, they use a lighter mobile interface on their phones or tablets to check their membership status, upcoming classes, and payment history. This device connects to the Application Server over Wi-Fi or mobile data.
- **Application Server:** This is where the real work happens. All the business logic — authentication, equipment tracking, maintenance processing, billing, trainer scheduling, and payroll — runs here. It receives requests from both the desktop and mobile clients and handles them through a REST API. It is also the only node that directly speaks to the database.
- **Database Server:** This node stores everything permanently — member records, equipment logs, maintenance history, payment data, trainer timetables, and staff payroll information. We deliberately kept it isolated so that only the Application Server can access it, which adds a layer of security.

Communication

- Personal Computer talks to the Application Server over HTTPS / REST through the gym's LAN or Wi-Fi.
- Mobile Device also reaches the Application Server over HTTPS / REST, either through Wi-Fi or mobile data.
- The Application Server connects to the Database Server internally over TCP, using whichever database driver is appropriate (MySQL or MongoDB).

Deployment Diagram Description

The deployment diagram represents the physical architecture of the **Gym Management System** and shows how the system components are distributed across different hardware devices. The **Personal Computer** is mainly used by the administrator to manage gym operations such as equipment tracking, maintenance scheduling, billing management, and trainer management. It contains components like the admin dashboard and different management modules that allow the administrator to control system activities. The **Mobile Device** represents the platform used by gym members to access basic services through the gym application. It provides a user interface that allows members to interact with the system, view information, and access gym-related services conveniently.

Both the personal computer and mobile device communicate with the **Application Server**, which contains the core logic of the system. The application server handles authentication, processes maintenance and billing operations, and generates reports when required. It acts as the central processing unit of the system.

The **Database Server** stores all important information such as equipment details, maintenance records, billing data, staff payroll information, and trainer schedules. The application server retrieves and updates this data whenever necessary. This deployment structure ensures proper organization of system components and supports efficient communication between the user devices, application server, and database.

PRODUCT METRICS

GYM MANAGEMENT SYSTEM

Metrics for the Requirements Model

Function Point Analysis (FPA)

the system is divided into five types of **functional components**:

External Inputs, External Outputs, External Inquiries, Internal Logical Files, and External Interface Files.

External Inputs (EI)

These are the inputs given by the user to the system.

- Adding new equipment (Simple)
- Updating equipment status (Simple)
- Raising maintenance requests (Simple)
- Entering billing details (Simple)
- Adding trainer information (Average)
- Processing payroll (Average)

So, total External Inputs = 6

External Outputs (EO)

These are the outputs generated by the system.

- Maintenance reports (Simple)
- Billing receipts (Simple)
- Equipment status reports (Simple)
- Payroll reports (Average)
- Trainer schedule reports (Average)

So, total External Outputs = 5

External Inquiries (EQ)

These are simple queries where the user requests information.

- Viewing equipment status (Simple)
- Checking maintenance status (Simple)
- Viewing billing history (Average)
- Checking trainer availability (Average)

So, total External Inquiries = 4

Internal Logical Files (ILF)

These are the data files maintained within the system.

- Equipment database (Average)
- Maintenance records (Average)
- Billing records (Average)

- Trainer details (Complex)
- Staff payroll data (Complex)

So, total ILFs = 5

External Interface Files (EIF)

These are external systems used by the application.

- Payment gateway system (Average)

So, total EIFs = 1

Step 1: Unadjusted Function Points (UFP)

Component	Count	Complexity Split	Weight	Calculation	Total
EI	6	4 Simple, 2 Average	3, 4,6	$(4 \times 3) + (2 \times 4)$	$12 + 8 = 20$
EO	5	3 Simple, 2 Average	4, 5,7	$(3 \times 4) + (2 \times 5)$	$12 + 10 = 22$
EQ	4	2 Simple, 2 Average	3, 4,6	$(2 \times 3) + (2 \times 4)$	$6 + 8 = 14$
ILF	5	3 Average, 2 Complex	7,10, 15	$(3 \times 10) + (2 \times 15)$	$30 + 30 = 60$
EIF	1	1 Average	8,7,10	(1×7)	7

UFP Calculation

$$\text{UFP} = 20 + 22 + 14 + 60 + 7 = 123$$

Step 2: Complexity Adjustment Factor (CAF)

Each of the following system characteristics is rated from 0 (No Influence) to 5 (Essential) based on how much it affects the system.

CAF Rating Table

No	Question	Rating (0–5)	Justification
1	Does the system require reliable backup and recovery?	4	Important because gym data like billing and records should not be lost.
2	Are specialized data communications required to transfer information to or from the application?	3	Normal client-server communication is needed, but nothing highly specialized.
3	Are there distributed processing functions?	2	Basic distribution between client and server, not highly complex.
4	Is performance critical?	3	Moderate importance as users expect quick responses.
5	Will the system run in an existing, heavily utilized operational environment?	4	Gym systems are used frequently by staff and admins daily.
6	Does the system require online data entry?	5	Most operations like billing, maintenance, and entries are done online.
7	Does the online data entry require the input transaction to be built over multiple screens or operations?	3	Some processes (like billing or payroll) involve multiple steps.
8	Are the ILFs updated online?	5	Databases like equipment, billing, and payroll are updated in real-time.
9	Are the inputs, outputs, files, or inquiries complex?	3	Moderate complexity with multiple modules involved.
10	Is the internal processing complex?	3	Some logic like billing and payroll adds moderate complexity.
11	Is the code designed to be reusable?	2	Basic level of reusability, not highly emphasized.
12	Are conversion and installation included in the design?	2	Minimal setup and installation requirements.
13	Is the system designed for multiple installations in different organizations?	2	Can be reused, but currently focused on a single gym.
14	Is the application designed to facilitate change and ease of use by the user?	4	User-friendly design and future updates are considered.

Sum of Factors

$$F_i = 4 + 3 + 2 + 3 + 4 + 5 + 3 + 5 + 3 + 3 + 2 + 2 + 2 + 4$$
$$F_i = 45$$

CAF Calculation

$$CAF = 0.65 + (0.01 \times 45)$$
$$CAF = 0.65 + 0.45 = 1.10$$

Step3: Multiply UFP and CAF

$$FP = UFP \times CAF$$
$$FP = 123 \times 1.10$$
$$= 135.3$$

FUNCTION POINT = 135.3

PART -02

Interpretation

The final value of around 135.3 function points shows that the Gym Management System has a moderate level of functionality.

If we assume that 1 function point requires around 10 hours of development, then:

Total effort $\approx 135.3 \times 10 = \mathbf{1353 \text{ hours}}$

If the development cost is ₹1000 per hour, then:

Estimated cost $\approx \mathbf{₹13,53,000 \text{ hours}}$

If we assume that 1 function point gets around 2 errors, then:

no of errors = **270.6 errors**

If we assume that 1 function point takes 10 lines of code , then:

No of lines of code = **1353 lines**

Empirical Estimation Models

-COCOMO Model

A) Basic COCOMO Model

The project has 45 methods assuming that each method takes approximately 1000 lines of code, this project has 45KLOC

Mode	Project size	Nature of Project	Innovation	Deadline of the project	Development Environment
Organic	Typically 2-50 KLOC	Small size project, experienced developers in the familiar environment. For example, pay roll, inventory projects etc.	Little	Not tight	Familiar & In house

Since we have 45KLOC, we categorise it as organic model

The values we use for organic model

Software Project	A	B	C	D
Organic	2.4	1.05	2.5	0.38
Semi Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

The following formulas are used to calculate effort and time estimates.

$$\text{Effort (E)} = A * (\text{KLOC})^B$$

$$= 2.4 * 45^{1.05}$$

$$= 136.48$$

$$\text{Time (T)} = C * (\text{Effort})^D$$

$$= 2.5 * 136.48^{0.38}$$

$$= 16.19$$

This project needs 16.19 months to be completed

$$\text{Persons required} = \text{Effort} / \text{Time}$$

$$= 136.48 / 16.19$$

$$= 8.42$$

This project needs approximately 9 persons per month

B) Intermediate COCOMO Model

Rating cost drivers for the calculation

(i) Product Attributes

Factor	Rating	Value
Required software reliability	High	1.15
Size of application database	Nominal	1.00
Product complexity	High	1.15

(ii) Hardware Attributes

Factor	Rating	Value
Run-time performance constraints	Nominal	1.00
Memory constraints	Nominal	1.00
Virtual machine volatility	Low	0.87
Required turnabout time	Nominal	1.00

(iii) Personnel Attributes

Factor	Rating	Value
Analyst capability	High	0.86
Software engineering capability	Nominal	1.00
Application experience	Nominal	1.00
Virtual machine experience	Low	1.10
Programming language experience	Nominal	1.00

(iv) Project Attributes

Factor	Rating	Value
Use of software tools	High	0.91
Software engineering methods	Nominal	1.00
Required development schedule	Nominal	1.00

EAF Calculation

$$EAF = 1.15 \times 1.00 \times 1.15 \times 1.00 \times 1.00 \times 0.87 \times 1.00 \times 0.86 \times 1.00 \times 1.00 \times 1.10 \times 1.00 \times 0.91 \times 1.00 \times 1.00$$

$$EAF = 1.09$$

Rating values

Cost drivers Number	Ratings					
	Very low	Low	Nominal	High	Very high	Extra high
1	0.75	0.88	1.00	1.15	1.40	-
2	-	0.94	1.00	1.08	1.16	-
3	0.70	0.85	1.00	1.15	1.30	1.65
4	-	-	1.00	1.11	1.30	1.66
5	-	-	1.00	1.06	1.21	1.56
6	-	0.87	1.00	1.15	1.30	-
7	-	0.87	1.00	1.07	1.15	-
8	1.46	1.19	1.00	0.86	0.71	-
9	1.29	1.13	1.00	0.91	0.82	-
10	1.42	1.17	1.00	0.86	0.70	-
11	1.21	1.10	1.00	0.90	-	-
12	1.14	1.07	1.00	0.95	-	-
13	1.24	1.10	1.00	0.91	0.82	-
14	1.24	1.10	1.00	0.91	0.83	-
15	1.23	1.08	1.00	1.04	1.10	-

Formulas to calculate

Effort (E) = $(A * (KLOC)^B) * EAF$.

Time (T) = $C * (Effort)^D$

Persons required = Effort / Time

The values we are using:

Since the project has 45 KLOC, it is considered as organic model and the values are taken accordingly

Software Project	A	B	C	D
Organic	3.2	1.05	2.5	0.38
Semi Detached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32

Calculations

Effort (E) = $(A * (KLOC)^B) * EAF$.

= $3.2 * 45^{1.05} * 1.09$

= **189.86**

Time (T) = $C * (Effort)^D$

= $2.5 * 189.86^{0.38}$

= **18.35**

The project approximately takes 18 months of time

$$\begin{aligned}\text{Persons required} &= \text{Effort} / \text{Time} \\ &= 189.86/18.35 \\ &= \mathbf{10.34}\end{aligned}$$

The project approximately takes 10 people per month